FINAL ImpactStudio FullSystem_v3

Partner-ready Colab workflow for **one-pass Run All** (and a few manual access granting) to launch the final Gradio UI.

Required Before Running

1. In Google Drive, open the shared project folder `FINAL_SYSTEM` and click **Add shortcut to Drive**.
2. Ensure shortcut is visible under **My Drive** (recommended name: `FINAL_SYSTEM`).
3. Keep this structure in the same shortcut folder:
   - `FINAL_ImpactStudio_FullSystem_v3.ipynb`
   - `my_finetuned_llm/` (contains `adapter_config.json` and adapter files)

This version auto-resolves adapter path from Drive shortcut.

Run Guide (Partner)

Recommended

1. In Colab: **Runtime -> Change runtime type -> GPU**
   - Runtime Type: Python 3
   - T4/A100 both work
   - High-RAM can be selected
2. On the top right side, click `Connect`.
3. Click `Run all` (from top menu).
4. When prompted, enter:
   - `GOOGLE_API_KEY`
   - `HF_TOKEN`
5. Wait until `STEP 14.1` prints Gradio launch URL, then open it.
   - There are two pre-made prompts for the orchestrator routing to each agent.

Notes

- No manual path edits are required in this v3.
- We have fallback key preserved in the key cell comment.
- If UI does not appear, re-run `STEP 14` and `STEP 14.1`.

Workflow Map

This notebook keeps the existing v2 logic and only reorganizes execution clarity for partner handoff:

- Section A/B: boot, dependencies, routing core
- Section C: Drive mount + automatic adapter path resolution
- Section D: Impact/RAG backend

- Section E: Unified Gradio UI launch

Goal: **Run all once** and get final UI without too many manual path tweaking.

Section A - Boot & Environment

Dependencies, imports, credentials, and basic checkpoints.

```
# ============================================================
# STEP 0: Progress Checkpoint
# ============================================================
print("[STEP 0] Notebook loaded.")
print("[STEP 0.1] Next: install dependencies.")
```

```
[STEP 0] Notebook loaded.
[STEP 0.1] Next: install dependencies.
```

```
# ============================================================
# STEP 1: Install Dependencies (copy from existing notebooks)
# ============================================================
print("[STEP 1] Installing dependencies...")

!pip -q install -U google-genai gradio
!pip -q install -U google-api-python-client google-auth-httplib2
google-auth-oauthlib
!pip -q install -U unsloth unsloth-zoo modelscope peft accelerate
bitsandbytes huggingface_hub python-docx PyPDF2
!pip -q install -U "sentence-transformers>=3.0.1" "faiss-cpu>=1.8.0"
"langchain-text-splitters>=0.3.0" "pypdf>=4.2.0"
"transformers>=4.45.0"

print("[STEP 1.1] Install complete.")
```

```
[STEP 1] Installing dependencies...
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 52.0/52.0 kB 4.3 MB/s eta
0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 724.4/724.4 kB 28.7 MB/s eta
0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 24.2/24.2 MB 61.1 MB/s eta
0:00:00:00:010:01m
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 58.3/58.3 kB 6.7 MB/s eta
0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 14.8/14.8 MB 118.2 MB/s eta
0:00:0000:0100:01
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 69.7/69.7 kB 6.9 MB/s eta
0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 43.3/43.3 kB 4.6 MB/s eta
0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 44.0/44.0 kB 4.4 MB/s eta
0:00:00
```

```
━━━
```

```python
# ==============================================================
# STEP 2: Imports + Runtime Flags
# ==============================================================
print("[STEP 2] Importing libraries...")

import os
import re
import torch
import getpass
import gradio as gr
from threading import Lock
from types import SimpleNamespace

from google import genai
from unsloth import FastLanguageModel
from peft import PeftModel

os.environ["UNSLOTH_DISABLE_LOG_STATS"] = "1"
os.environ.setdefault("UNSLOTH_USE_MODELSCOPE", "1")

print("[STEP 2.1] Imports ready.")
```

```
[STEP 2] Importing libraries...
 Unsloth: Will patch your computer to enable 2x faster free
finetuning.
 Unsloth Zoo will now patch everything to make training faster!
[STEP 2.1] Imports ready.
```

NEXT MANUAL: Enter API KEYs

```python
# ==============================================================
# STEP 3A: Keys / Tokens Input (same style as original)
# ==============================================================
print("[STEP 3A] Loading credentials input...")

# Keep original-style input behavior in one cell.
if not os.getenv("GOOGLE_API_KEY"):
    os.environ["GOOGLE_API_KEY"] = getpass.getpass("Enter
GOOGLE_API_KEY: ")
    # AIzaSyAFxYzzzJKLGoxmw5QtQA8lVV__vS5TpUY

if not os.getenv("HF_TOKEN"):
    os.environ["HF_TOKEN"] = getpass.getpass("Enter HF_TOKEN: ")
    # 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

print("[STEP 3A.1] Raw GOOGLE_API_KEY captured in env.")
print("[STEP 3A.2] HF_TOKEN captured in env.")
```

```
[STEP 3A] Loading credentials input...
[STEP 3A.1] Raw GOOGLE_API_KEY captured in env.
[STEP 3A.2] HF_TOKEN captured in env.
```

NEXT MANUAL: STEP 7

Since the code runs quickly, the next manual step is **Step 7**. This step requires Colab to connect to your **Google Drive**. I move the text cell here since it'll be more convenient.

- Please log in to your Google Drive
- Grant the required permissions when prompted.
- Make sure the **FINAL_SYSTEM** folder **shortcut** is located in your **My Drive** (not only in "Shared with me").

```
# ============================================================
# STEP 3B: GOOGLE_API_KEY Extraction / Normalization
# ============================================================
print("[STEP 3B] Extracting/normalizing GOOGLE_API_KEY...")

import re

# Toggle: show full key values for debugging.
SHOW_FULL_GOOGLE_KEY = True

def _normalize_google_api_key(raw: str) -> str:
    raw = (raw or "").strip()
    # Extract AIza token even if user pasted comments/extra text.
    m = re.search(r"AIza[0-9A-Za-z_-]{20,}", raw)
    if m:
        return m.group(0).strip()
    return raw

raw_google_input = os.getenv("GOOGLE_API_KEY", "")
normalized_google = _normalize_google_api_key(raw_google_input)

# Overwrite runtime key with extracted token
os.environ["GOOGLE_API_KEY"] = normalized_google
GOOGLE_API_KEY = normalized_google

if SHOW_FULL_GOOGLE_KEY:
    print("[STEP 3B.1] Raw GOOGLE_API_KEY:", raw_google_input)
    print("[STEP 3B.2] Extracted GOOGLE_API_KEY:", normalized_google)
else:
    print("[STEP 3B.1] Raw GOOGLE_API_KEY length:",
len(raw_google_input or ""))
    print("[STEP 3B.2] Extracted GOOGLE_API_KEY prefix:",
(normalized_google[:8] + "...") if normalized_google else "(empty)")

if not normalized_google or not normalized_google.startswith("AIza"):
    print("[STEP 3B WARN] Extracted GOOGLE_API_KEY format looks
```

```
unusual.")
else:
    print(f"[STEP 3B.3] GOOGLE_API_KEY ready (prefix): 
{normalized_google[:8]}...")
```

```
[STEP 3B] Extracting/normalizing GOOGLE_API_KEY...
[STEP 3B.1] Raw GOOGLE_API_KEY: 
AIzaSyAFxYzzzJKLGoxmw5QtQA8lVV__vS5TpUY
[STEP 3B.2] Extracted GOOGLE_API_KEY: 
AIzaSyAFxYzzzJKLGoxmw5QtQA8lVV__vS5TpUY
[STEP 3B.3] GOOGLE_API_KEY ready (prefix): AIzaSyAF...
```

Section B - Routing Core

Router initialization and safe routing behavior.

```
# ============================================================
# STEP 4: RequestOrchestrator (copied from Orchestrator_Pipeline)
# ============================================================
print("[STEP 4] Defining orchestrator class...")

class RequestOrchestrator:
    """
    Orchestrator that determines which agent should handle a user's 
request.
    Routes between Impact Agent and Script Reviewer.
    """

    IMPACT_AGENT = "impact agent"
    SCRIPT_REVIEWER = "script reviewer"

    def __init__(self, api_key: str, model_name: str = "gemini-2.5-
flash"):
        self.client = genai.Client(api_key=api_key)
        self.model_name = model_name
        print(f"[STEP 4.1] Orchestrator initialized with 
{model_name}")

    def _create_routing_prompt(self, user_prompt: str) -> str:
        system_prompt = f"""You are a routing system that determines 
which agent should handle a user's request.

You have TWO agents available:

1. **Impact Agent**: Handles questions about:
   - Impact, outcomes, effects, consequences, results
   - Benefits, changes, influence of actions/policies/programs
   - Analysis of societal, environmental, or business impacts
   - "What if" scenarios and their implications
```

```
2. **Script Reviewer**: Handles questions about:
   - Reviewing, analyzing, or critiquing scripts (movie, TV, theater)
   - Code review and programming help
   - Document review and editing
   - Writing feedback and improvements

USER QUERY: {user_prompt}

Based on the user's query, determine which agent should handle this
request.

RESPOND WITH ONLY ONE OF THESE TWO OPTIONS (exactly as written):
- "impact agent"
- "script reviewer"

Your response should contain ONLY the agent name, nothing else."""
        return system_prompt

    def route_request(self, user_prompt: str) -> str:
        prompt = self._create_routing_prompt(user_prompt)
        try:
            response =
self.client.models.generate_content(model=self.model_name,
contents=prompt)
            decision = (getattr(response, "text", "") or
"").strip().lower()
            if self.IMPACT_AGENT in decision:
                return self.IMPACT_AGENT
            elif self.SCRIPT_REVIEWER in decision:
                return self.SCRIPT_REVIEWER
            else:
                print(f"[STEP 4 WARN] Unexpected routing response:
{decision}. Using fallback.")
                return self._fallback_routing(user_prompt)
        except Exception as e:
            print(f"[STEP 4 WARN] Routing error: {str(e)}. Using
fallback.")
            return self._fallback_routing(user_prompt)

    def _fallback_routing(self, user_prompt: str) -> str:
        user_lower = user_prompt.lower()

        script_keywords = ["script", "review", "code", "analyze",
"critique",
                           "feedback", "document", "written", "check",
"screenplay",
                           "program", "function", "bug", "error",
"syntax"]
```

```
        impact_keywords = ["impact", "effect", "outcome", "result",
"consequence",
                           "benefit", "change", "influence", "affect",
"what if",
                           "implications", "happens", "caused by"]

        script_score = sum(1 for keyword in script_keywords if keyword
in user_lower)
        impact_score = sum(1 for keyword in impact_keywords if keyword
in user_lower)

        if script_score > impact_score:
            return self.SCRIPT_REVIEWER
        else:
            return self.IMPACT_AGENT

print("[STEP 4.2] Orchestrator class ready.")
```

```
[STEP 4] Defining orchestrator class...
[STEP 4.2] Orchestrator class ready.
```

```
# ==============================================================
# STEP 5: Default User Prompt (for UI prefill)
# ==============================================================
print("[STEP 5] Setting default prompt/context for Gradio UI...")

# This default text is prefilled in the input box.
# You can edit it in UI before clicking Analyze.
USER_PROMPT = "Please review my script opening and tell me what to
improve."
ADDITIONAL_CONTEXT = "Focus on clarity, pacing, and emotional impact."

# Keep for backward compatibility with STEP 10 one-shot test.
# In Gradio flow, users upload files directly (no Drive file path
needed here).
UPLOADED_FILE_PATH = None

print("[STEP 5.1] Default USER_PROMPT set.")
print("[STEP 5.2] Default CONTEXT set.")
print("[STEP 5.3] File will come from Gradio upload.")
```

```
[STEP 5] Setting default prompt/context for Gradio UI...
[STEP 5.1] Default USER_PROMPT set.
[STEP 5.2] Default CONTEXT set.
[STEP 5.3] File will come from Gradio upload.
```

```
# ==============================================================
# OPTIONAL: Gemini Connectivity Quick Check
# ==============================================================
print("[OPTIONAL] Skip this cell if routing is already working.")
```

```
print("[OPTIONAL] This cell is intentionally lightweight to avoid long
hangs.")
```

```
[OPTIONAL] Skip this cell if routing is already working.
[OPTIONAL] This cell is intentionally lightweight to avoid long hangs.
```

Legacy Step (Skip)

This old Step 6 cell is deprecated. Use the next code cell: **STEP 6: Route Prompt with Orchestrator (API timeout + fallback)**.

```
# ============================================================
# STEP 6: Initialize Router (routing executes only on Analyze click)
# ============================================================
print("[STEP 6] Initializing orchestrator and safe_route...")

ROUTER_MODEL = os.getenv("ROUTER_MODEL", "gemini-2.5-flash-lite")
orchestrator =
RequestOrchestrator(api_key=os.environ["GOOGLE_API_KEY"],
model_name=ROUTER_MODEL)
print(f"[STEP 6.0] Router model: {ROUTER_MODEL}")


def keyword_route(prompt: str):
    text = (prompt or "").lower()

    script_keywords = [
        "script", "review", "screenplay", "dialogue", "plot",
"character",
        "code", "bug", "function", "error", "syntax", "refactor",
"document"
    ]
    impact_keywords = [
        "impact", "outcome", "consequence", "benefit", "harm",
"society",
        "community", "ethics", "policy", "what if", "implication"
    ]

    script_score = sum(1 for k in script_keywords if k in text)
    impact_score = sum(1 for k in impact_keywords if k in text)

    if script_score > 0 and impact_score == 0:
        return orchestrator.SCRIPT_REVIEWER, script_score,
impact_score
    if impact_score > 0 and script_score == 0:
        return orchestrator.IMPACT_AGENT, script_score, impact_score

    return None, script_score, impact_score
```

```
def safe_route(prompt: str, timeout_sec: int = 12):
    # 1) Fast keyword route first (always prompt-based)
    kw_agent, s_score, i_score = keyword_route(prompt)
    if kw_agent is not None:
        return kw_agent, f"keyword(script={s_score},impact={i_score})"

    # 2) LLM route only when keyword route is ambiguous
    try:
        routing_prompt = orchestrator._create_routing_prompt(prompt)
        response = orchestrator.client.models.generate_content(
            model=orchestrator.model_name,
            contents=routing_prompt,
        )
        decision = (getattr(response, "text", "") or
"").strip().lower()

        if orchestrator.IMPACT_AGENT in decision:
            return orchestrator.IMPACT_AGENT, "llm"
        if orchestrator.SCRIPT_REVIEWER in decision:
            return orchestrator.SCRIPT_REVIEWER, "llm"

        fb = orchestrator._fallback_routing(prompt)
        return fb,
f"fallback_format(script={s_score},impact={i_score})"

    except Exception as e:
        fb = orchestrator._fallback_routing(prompt)
        return fb, f"fallback_error:{type(e).__name__}
(script={s_score},impact={i_score})"


print("[STEP 6.1] safe_route is ready.")
print("[STEP 6.2] Routing runs in STEP 11 when Analyze is clicked.")
```

```
[STEP 6] Initializing orchestrator and safe_route...
[STEP 4.1] Orchestrator initialized with gemini-2.5-flash-lite
[STEP 6.0] Router model: gemini-2.5-flash-lite
[STEP 6.1] safe_route is ready.
[STEP 6.2] Routing runs in STEP 11 when Analyze is clicked.
```

Section C - Drive Mount & Fine-tuned Model (FINAL_SYSTEM Shortcut)

This section validates `MyDrive/FINAL_SYSTEM` and checks `my_finetuned_llm` adapter files.

```
# ============================================================
# STEP 7: Mount Google Drive
# ============================================================
```

```
print("[STEP 7] Mounting Google Drive...")

import os

try:
    from google.colab import drive
    drive.mount('/content/drive')
    print("[STEP 7.1] Drive mounted (or already mounted).")
except Exception as e:
    print("[STEP 7 ERROR] Could not mount Drive:", e)
    raise

print("[STEP 7.1] Tip: Ensure shared folder was added as shortcut
under MyDrive.")
```

```
[STEP 7] Mounting Google Drive...
Drive already mounted at /content/drive; to attempt to forcibly
remount, call drive.mount("/content/drive", force_remount=True).
[STEP 7.1] Drive mounted (or already mounted).
[STEP 7.1] Tip: Ensure shared folder was added as shortcut under
MyDrive.
```

```
# ============================================================
# STEP 7.2: Validate FINAL_SYSTEM shortcut + my_finetuned_llm
# ============================================================
import os

MYDRIVE_ROOT = "/content/drive/MyDrive"
PROJECT_ROOT = os.path.join(MYDRIVE_ROOT, "FINAL_SYSTEM")
ADAPTER_DIR = os.path.join(PROJECT_ROOT, "my_finetuned_llm")

print("[STEP 7.2] Checking project shortcut:", PROJECT_ROOT)
if not os.path.isdir(PROJECT_ROOT):
    raise FileNotFoundError(
        "FINAL_SYSTEM shortcut folder not found in MyDrive. "
        "Please add the shared folder shortcut to MyDrive and re-run
STEP 7."
    )

print("[STEP 7.2] FINAL_SYSTEM exists: True")
print("[STEP 7.2] FINAL_SYSTEM top-level contents:")
for name in sorted(os.listdir(PROJECT_ROOT)):
    full = os.path.join(PROJECT_ROOT, name)
    tag = "<DIR>" if os.path.isdir(full) else "<FILE>"
    print(f"  {tag} {name}")

print("\n[STEP 7.2] Checking adapter folder:", ADAPTER_DIR)
if not os.path.isdir(ADAPTER_DIR):
    raise FileNotFoundError(
        "my_finetuned_llm folder not found under FINAL_SYSTEM. "
```

```
        "Expected path:
/content/drive/MyDrive/FINAL_SYSTEM/my_finetuned_llm"
    )

required_files = [
    "adapter_config.json",
    "adapter_model.safetensors",
]
optional_files = [
    "README.md",
]

missing_required = [f for f in required_files if not
os.path.isfile(os.path.join(ADAPTER_DIR, f))]

print("[STEP 7.2] Required adapter files check:")
for f in required_files:
    ok = os.path.isfile(os.path.join(ADAPTER_DIR, f))
    print(f"  - {f}: {'OK' if ok else 'MISSING'}")

print("[STEP 7.2] Optional files check:")
for f in optional_files:
    ok = os.path.isfile(os.path.join(ADAPTER_DIR, f))
    print(f"  - {f}: {'FOUND' if ok else 'NOT FOUND'}")

print("\n[STEP 7.2] All files inside my_finetuned_llm:")
for name in sorted(os.listdir(ADAPTER_DIR)):
    full = os.path.join(ADAPTER_DIR, name)
    tag = "<DIR>" if os.path.isdir(full) else "<FILE>"
    print(f"  {tag} {name}")

if missing_required:
    raise FileNotFoundError(
        "Missing required adapter files in my_finetuned_llm: " + ",
".join(missing_required)
    )

print("\n[STEP 7.2] Adapter directory validation passed.")
print("[STEP 7.3] ADAPTER_DIR set to:", ADAPTER_DIR)
```

```
[STEP 7.2] Checking project shortcut:
/content/drive/MyDrive/FINAL_SYSTEM
[STEP 7.2] FINAL_SYSTEM exists: True
[STEP 7.2] FINAL_SYSTEM top-level contents:
  <FILE> FINAL_ImpactStudio_FullSystem_v3.ipynb
  <DIR> my_finetuned_llm
  <FILE> 【Deprecated】FINAL_ImpactStudio_FullSystem_v1.ipynb

[STEP 7.2] Checking adapter folder:
/content/drive/MyDrive/FINAL_SYSTEM/my_finetuned_llm
```

```
[STEP 7.2] Required adapter files check:
  - adapter_config.json: OK
  - adapter_model.safetensors: OK
[STEP 7.2] Optional files check:
  - README.md: FOUND

[STEP 7.2] All files inside my_finetuned_llm:
  <FILE> README.md
  <FILE> adapter_config.json
  <FILE> adapter_model.safetensors
  <FILE> chat_template.jinja
  <FILE> special_tokens_map.json
  <FILE> tokenizer.json
  <FILE> tokenizer_config.json

[STEP 7.2] Adapter directory validation passed.
[STEP 7.3] ADAPTER_DIR set to:
/content/drive/MyDrive/FINAL_SYSTEM/my_finetuned_llm
```

```python
# ============================================================
# STEP 8: Load Base Model + LoRA (copied from usemodelkaggle)
# ============================================================
print("[STEP 8] Loading fine-tuned model stack...")

BASE_ID = "unsloth/meta-llama-3.1-8b-instruct-unsloth-bnb-4bit"
HF_TOKEN = os.getenv("HF_TOKEN")
if not HF_TOKEN:
    raise ValueError("Missing HF_TOKEN. Set it before running this
cell.")

try:
    model, tokenizer = FastLanguageModel.from_pretrained(
        BASE_ID,
        max_seq_length=2048,
        load_in_4bit=True,
        token=HF_TOKEN,
        disable_log_stats=True,
    )
except TimeoutError:
    os.environ["UNSLOTH_USE_MODELSCOPE"] = "1"
    model, tokenizer = FastLanguageModel.from_pretrained(
        BASE_ID,
        max_seq_length=2048,
        load_in_4bit=True,
        token=HF_TOKEN,
        disable_log_stats=True,
    )

model = PeftModel.from_pretrained(model, ADAPTER_DIR)
FastLanguageModel.for_inference(model)
```

```
model.eval()

print("[STEP 8.1] Model + LoRA ready.")
```

```
[STEP 8] Loading fine-tuned model stack...
==((====))==  Unsloth 2026.2.1: Fast Llama patching. Transformers:
5.2.0.
   \\   /|    Tesla T4. Num GPUs = 1. Max memory: 14.563 GB. Platform:
Linux.
O^O/ \_/ \    Torch: 2.10.0+cu128. CUDA: 7.5. CUDA Toolkit: 12.8.
Triton: 3.6.0
\        /    Bfloat16 = FALSE. FA [Xformers = 0.0.35. FA2 = False]
 "-____-"     Free license: http://github.com/unslothai/unsloth
Unsloth: Fast downloading is enabled - ignore downloading bars which
are red colored!
```


```
{"model_id":"684cc94838c7436e81522a4aa1de3dea","version_major":2,"vers
ion_minor":0}

{"model_id":"0c105925be5a468daf08e4b6796b1bd1","version_major":2,"vers
ion_minor":0}

{"model_id":"ec39a6d9667c451e95e32dd1c85dce58","version_major":2,"vers
ion_minor":0}

{"model_id":"15fcf758a0a944d1862ab1fc0d2c5231","version_major":2,"vers
ion_minor":0}

{"model_id":"feac736b581d47679b92b51aa07cc90c","version_major":2,"vers
ion_minor":0}

{"model_id":"6d745dbe17a141bb903af455e0da1d3b","version_major":2,"vers
ion_minor":0}

{"model_id":"3ab00e5fcaf14eebb2d4d25dbade4267","version_major":2,"vers
ion_minor":0}
```


```
Unsloth: Will load unsloth/meta-llama-3.1-8b-instruct-unsloth-bnb-4bit
as a legacy tokenizer.

[STEP 8.1] Model + LoRA ready.
```

POSSIBLE MANUAL: If you got stuck here

If you can only see a red bar for model.safetensors, and nothing else, then:

- **Runtime -> Interrupt execution -> Run all when interrupted**

If execution cannot be interrupted, then:

- **Runtime -> Restart session and run all**

```python
# ================================================================
# STEP 9: Inference Function (copied from usemodelkaggle, non-Gradio)
# ================================================================
print("[STEP 9] Defining evaluator...")

import io

tok = tokenizer
mdl = model
mdl.eval()

device = "cuda" if torch.cuda.is_available() else "cpu"
gen_lock = Lock()
history = []

try:
    from transformers.utils import logging as hf_logging
    hf_logging.set_verbosity_error()
except Exception:
    pass

# Reviewer output cleanup switch.
# True: trim trailing dash-noise/artifacts in parsed text.
# False: keep full untrimmed text for inspection.
REVIEW_TRUNCATE_NOISE = True

_DRIVE_API = None


def _init_drive_api():
    global _DRIVE_API
    if _DRIVE_API is not None:
        return _DRIVE_API

    try:
        from google.colab import auth
        auth.authenticate_user()
    except Exception:
        # Non-Colab env may still have ADC credentials.
        pass

    try:
        import google.auth
        from googleapiclient.discovery import build
        creds, _ =
google.auth.default(scopes=["https://www.googleapis.com/auth/drive.rea
donly"])
        _DRIVE_API = build("drive", "v3", credentials=creds,
cache_discovery=False)
        return _DRIVE_API
    except Exception as e:
```

```
        print(f"[STEP 9 WARN] Drive API init failed: {e}")
        return None


def _extract_drive_id_from_url(url: str) -> str:
    if not url:
        return ""
    patterns = [
        r"/d/e/([a-zA-Z0-9_-]+)",
        r"/d/([a-zA-Z0-9_-]+)",
        r"[?&]id=([a-zA-Z0-9_-]+)",
    ]
    for pat in patterns:
        m = re.search(pat, url)
        if m:
            return m.group(1)
    return ""


def _read_google_shortcut_meta(path: str):
    """
    Read .gdoc/.gsheet/.gslides shortcut metadata.
    Supports JSON shortcut files and INI-style URL shortcuts.
    Returns (file_id, url).
    """
    url = ""
    file_id = ""

    raw = ""
    try:
        with open(path, "r", encoding="utf-8", errors="ignore") as f:
            raw = f.read()
    except Exception:
        return "", ""

    # Try JSON first
    try:
        meta = json.loads(raw)
        if isinstance(meta, dict):
            url = str(meta.get("url", "") or "")
            file_id = str(meta.get("doc_id", "") or "")

            resource_id = str(meta.get("resource_id", "") or "")
            if not file_id and resource_id:
                # examples: "document/FILE_ID" or "document:FILE_ID"
                if "/" in resource_id:
                    file_id = resource_id.split("/")[-1]
                elif ":" in resource_id:
                    file_id = resource_id.split(":")[-1]
```

```
            if not file_id:
                file_id = _extract_drive_id_from_url(url)

            if file_id:
                return file_id, url
    except Exception:
        pass

    # Fallback for non-JSON shortcut content
    # e.g. [InternetShortcut]
URL=https://docs.google.com/document/d/...
    m = re.search(r"(?im)^URL\s*=\s*(https?://\S+)", raw)
    if m:
        url = m.group(1).strip()
    else:
        m2 = re.search(r"https?://\S+", raw)
        if m2:
            url = m2.group(0).strip()

    if not file_id and url:
        file_id = _extract_drive_id_from_url(url)

    return file_id, url


def _google_mime_for_shortcut_ext(ext: str) -> str:
    ext = ext.lower()
    mapping = {
        ".gdoc": "application/vnd.google-apps.document",
        ".gsheet": "application/vnd.google-apps.spreadsheet",
        ".gslides": "application/vnd.google-apps.presentation",
        ".gdraw": "application/vnd.google-apps.drawing",
        ".gform": "application/vnd.google-apps.form",
        ".gsite": "application/vnd.google-apps.site",
    }
    return mapping.get(ext, "")


def _lookup_google_file_id_by_name(local_path: str):
    """
    Fallback when .g* shortcut metadata has no file id.
    Lookup by base filename in user's Drive.
    Returns (file_id, webViewLink).
    """
    svc = _init_drive_api()
    if svc is None:
        return "", ""

    stem = os.path.splitext(os.path.basename(local_path))[0]
    ext = os.path.splitext(local_path)[1].lower()
    mime = _google_mime_for_shortcut_ext(ext)
```

```
    safe_name = stem.replace("'", "\'")
    q = f"name = '{safe_name}' and trashed = false"
    if mime:
        q += f" and mimeType = '{mime}'"

    try:
        resp = svc.files().list(
            q=q,
            corpora="user",
            pageSize=10,
            fields="files(id,name,mimeType,webViewLink)",
        ).execute()
        files = resp.get("files", [])
        if files:
            return files[0].get("id", ""), files[0].get("webViewLink",
"")
    except Exception:
        pass

    # broad fallback: drop mime filter
    try:
        resp = svc.files().list(
            q=f"name = '{safe_name}' and trashed = false",
            corpora="user",
            pageSize=10,
            fields="files(id,name,mimeType,webViewLink)",
        ).execute()
        files = resp.get("files", [])
        if files:
            return files[0].get("id", ""), files[0].get("webViewLink",
"")
    except Exception:
        pass

    return "", ""


def _preferred_export_mimes(ext: str):
    ext = ext.lower()
    if ext == ".gsheet":
        return ["text/csv", "application/pdf", "text/plain"]
    if ext == ".gslides":
        return ["text/plain", "application/pdf"]
    if ext == ".gdraw":
        return ["application/pdf", "image/svg+xml", "text/plain"]
    if ext == ".gdoc":
        return ["text/plain", "application/pdf"]
    # generic .g* fallback
    return ["text/plain", "application/pdf", "text/csv"]
```

```
def _decode_export_bytes(data, mime_type: str):
    if isinstance(data, str):
        return data

    if mime_type == "application/pdf":
        import PyPDF2
        reader = PyPDF2.PdfReader(io.BytesIO(data))
        return "\n".join([page.extract_text() or "" for page in
reader.pages])

    for enc in ("utf-8", "latin-1"):
        try:
            return data.decode(enc, errors="ignore")
        except Exception:
            pass
    return str(data)


def read_google_workspace_file(path: str) -> str:
    ext = os.path.splitext(path)[1].lower()
    if not ext.startswith('.g'):
        return ""

    file_id, web_url = _read_google_shortcut_meta(path)

    # fallback by Drive lookup if shortcut meta didn't include file id
    if not file_id:
        looked_id, looked_url = _lookup_google_file_id_by_name(path)
        if looked_id:
            file_id = looked_id
            if not web_url:
                web_url = looked_url

    if not file_id:
        return f"(Google shortcut found but file id not found:
{path})"

    svc = _init_drive_api()
    if svc is None:
        return f"(Drive API not available for reading {path})"

    last_err = None
    for mime in _preferred_export_mimes(ext):
        try:
            payload = svc.files().export(fileId=file_id,
mimeType=mime).execute()
            text = _decode_export_bytes(payload, mime)
            if text and text.strip():
                return text
```

```
        except Exception as e:
            last_err = e

    # metadata fallback
    try:
        meta = svc.files().get(fileId=file_id,
fields="name,mimeType,webViewLink").execute()
        return (
            f"(Could not export content. name={meta.get('name')}
mime={meta.get('mimeType')} "
            f"link={meta.get('webViewLink', web_url)})"
        )
    except Exception:
        return f"(Could not export Google file content. Last error:
{last_err})"


def _clean_model_output(raw: str) -> str:
    text = (raw or "").replace("\r\n", "\n")
    text = text.replace("—", "-").replace("–", "-")
    text = re.sub(r"(?is)^.*?###\s*Response:\s*", "", text).strip()

    # Drop obvious repeated consecutive lines produced by sampling
loops.
    compact_lines = []
    prev_norm = ""
    repeat_count = 0
    for ln in text.split("\n"):
        norm = re.sub(r"\s+", " ", ln).strip().lower()
        if not norm:
            if compact_lines and compact_lines[-1] != "":
                compact_lines.append("")
            prev_norm = ""
            repeat_count = 0
            continue

        if norm == prev_norm:
            repeat_count += 1
            if repeat_count >= 1:
                continue
        else:
            repeat_count = 0

        compact_lines.append(ln.rstrip())
        prev_norm = norm

    cleaned = "\n".join(compact_lines).strip()
    cleaned = re.sub(r"\n{3,}", "\n\n", cleaned)
    return cleaned
```

```
def _truncate_reviewer_noise(text: str) -> str:
    src = (text or "").strip()
    if not src or not REVIEW_TRUNCATE_NOISE:
        return src

    # Pattern seen in model tails: repeated long dash groups like "---
--- ---".
    m = re.search(r"(?:\s[-]{2,}\s*){3,}", src)
    if m:
        return src[:m.start()].rstrip()
    return src


def _extract_first_review_block(text: str) -> str:
    data = (text or "").strip()
    if not data:
        return ""

    tagged = re.search(r"(?is)\[BEGIN_REVIEW\](.*?)\[END_REVIEW\]",
data)
    if tagged:
        data = tagged.group(1).strip()
    else:
        contamination_markers = [
            r"\nWait\s+for\s+further\s+instructions",
            r"\n###\s*Input\s*:",
            r"\n###\s*Instruction\s*:",
            r"\nIs\s+this\s+uplifting\?",
            r"\nInput\s*:",
        ]
        cut = len(data)
        for pat in contamination_markers:
            m = re.search(pat, data, re.I)
            if m:
                cut = min(cut, m.start())
        data = data[:cut].strip()

    data = _truncate_reviewer_noise(data)

    # Keep only up to the first complete overall rationale paragraph
if possible.
    m_overall = re.search(r"(?is)(?:^|\n)\s*(?:Overall\s*rationale|
Overall|Rationale)\s*[:\-]\s*(.+)", data)
    if m_overall:
        prefix = data[:m_overall.start()]
        tail = _truncate_reviewer_noise(m_overall.group(1).strip())
        stop = re.search(r"\n\s*\n", tail)
        first_para = tail[:stop.start()].strip() if stop else
tail.strip()
```

```
        data = (prefix + "\nOverall rationale: " + first_para).strip()

    return data


def _extract_section(text: str, labels, next_labels) -> str:
    label_pat = "|".join(re.escape(x) for x in labels)
    next_pat = "|".join(re.escape(x) for x in next_labels) if
next_labels else ""

    if next_pat:
        pattern = (
            rf"(?ims)(?:^|\n)\s*(?:#{{1,6}}\s*)?(?:[-*]\s*)?(?:
{label_pat})\s*[:\-]\s*"
            rf"(.+?)(?=\n\s*(?:#{{1,6}}\s*)?(?:[-*]\s*)?(?:
{next_pat})\s*[:\-]|\Z)"
        )
    else:
        pattern = (
            rf"(?ims)(?:^|\n)\s*(?:#{{1,6}}\s*)?(?:[-*]\s*)?(?:
{label_pat})\s*[:\-]\s*(.+)$"
        )

    m = re.search(pattern, text)
    return m.group(1).strip() if m else ""


def _dedupe_section_lines(block: str) -> str:
    if not block:
        return ""
    out = []
    seen = set()
    for ln in block.splitlines():
        norm = re.sub(r"[^a-z0-9]+", " ", ln.lower()).strip()
        if norm and norm in seen:
            continue
        if norm:
            seen.add(norm)
        out.append(ln.rstrip())
    return "\n".join(out).strip()


def _parse_score_to_int(score_text: str, fallback_text: str) -> int:
    cand = (score_text or "").strip()

    m = re.search(r"([1-5](?:\.\d+)?)", cand)
    if not m:
        m = re.search(r"([1-5](?:\.\d+)?)\s*(?:/|out of)\s*5",
fallback_text or "", re.I)
    if not m:
```

```
        return 1

    try:
        val = float(m.group(1))
        return max(1, min(5, int(round(val))))
    except Exception:
        return 1


def _normalize_verdict(verdict_text: str) -> str:
    text = (verdict_text or "").strip()
    m = re.search(r"\b(yes|no)\b", text, re.I)
    return m.group(1).capitalize() if m else ""


def _parsed_quality(parsed: dict) -> int:
    quality = 0
    for key in ["verdict", "score", "benefits", "risks", "overall"]:
        val = (parsed.get(key) or "").strip()
        if val:
            quality += 1
            if len(val) >= 40:
                quality += 1
    return quality


def format_output(raw):
    text = _extract_first_review_block(_clean_model_output(raw))

    verdict = _extract_section(
        text,
        ["Verdict", "Final Verdict"],
        ["Score", "Rating", "Benefits", "Risks", "Overall rationale",
"Overall", "Rationale", "Analysis"],
    )
    if not verdict:
        m_v = re.search(r"(?im)\bIs\s+this\s+uplifting\?\s*(yes|no)\
b", text)
        if not m_v:
            m_v = re.search(r"(?im)^\s*(yes|no)\s*$", text)
        if m_v:
            verdict = m_v.group(1).capitalize()

    score = _extract_section(
        text,
        ["Score", "Rating"],
        ["Benefits", "Risks", "Overall rationale", "Overall",
"Rationale", "Analysis"],
    )
    benefits = _extract_section(
        text,
```

```
        ["Benefits", "Strengths", "Upsides", "Positive impacts"],
        ["Risks", "Concerns", "Downsides", "Overall rationale",
"Overall", "Rationale", "Analysis"],
    )
    risks = _extract_section(
        text,
        ["Risks", "Concerns", "Downsides", "Negative impacts"],
        ["Overall rationale", "Overall", "Rationale", "Analysis"],
    )
    overall = _extract_section(
        text,
        ["Overall rationale", "Overall", "Rationale", "Analysis",
"Final analysis"],
        [],
    )

    # Fallback if label extraction fails but text exists.
    if not overall and text:
        overall = text

    verdict = verdict.replace("**", "").strip()
    score = score.replace("**", "").strip()
    benefits = _dedupe_section_lines(benefits)
    risks = _dedupe_section_lines(risks)
    overall = _dedupe_section_lines(overall)

    return {
        "verdict": verdict,
        "score": score,
        "benefits": benefits,
        "risks": risks,
        "overall": overall,
        "raw": text,
    }





# ===== Script Reviewer summary gate (long-input protection) =====
SCRIPT_SUMMARY_TRIGGER_TOKENS = 2048
SCRIPT_REVIEWER_MAX_PROMPT_TOKENS = 2048
SCRIPT_SUMMARY_TARGETS = ["900-1200", "500-700", "300-450"]
SCRIPT_SUMMARY_MODEL = os.getenv("SCRIPT_SUMMARY_MODEL", "gemini-3-
flash-preview")
SCRIPT_SUMMARY_MODEL_FALLBACK =
os.getenv("SCRIPT_SUMMARY_MODEL_FALLBACK", "gemini-2.5-flash-lite")

# Cache rejected key to avoid repeated API_KEY_INVALID spam in a
single runtime.
```

```
SUMMARY_GATE_REJECTED_KEY = None


def _current_google_api_key() -> str:
    return (globals().get("GOOGLE_API_KEY") or
os.getenv("GOOGLE_API_KEY") or "").strip()


def _is_google_api_key_invalid_error(err: Exception) -> bool:
    msg = str(err).lower()
    return (
        "api_key_invalid" in msg
        or "api key not valid" in msg
        or "invalid api key" in msg
        or ("invalid_argument" in msg and "api key" in msg)
    )


def _count_local_tokens(text: str) -> int:
    text = text or ""
    if not text:
        return 0

    try:
        enc = tok(text, add_special_tokens=False)
        ids = enc["input_ids"] if isinstance(enc, dict) else
enc.input_ids
        if ids and isinstance(ids[0], list):
            return len(ids[0])
        return len(ids)
    except Exception:
        try:
            return len(tok.encode(text))
        except Exception:
            return max(1, len(text) // 4)


def _truncate_to_tokens(text: str, max_tokens: int) -> str:
    if max_tokens <= 0:
        return ""
    try:
        enc = tok(text, add_special_tokens=False)
        ids = enc["input_ids"] if isinstance(enc, dict) else
enc.input_ids
        if ids and isinstance(ids[0], list):
            ids = ids[0]
        if len(ids) <= max_tokens:
            return text
        return tok.decode(ids[:max_tokens], skip_special_tokens=True)
    except Exception:
        return text[: max_tokens * 4]
```

```
def _summary_system_instruction(target_token_range: str) -> str:
    return f"""
Role: You are a Structural Script Analyst and Narrative Architect.
Task: Compress the provided raw script treatment into a "Uplift-Ready
Structural Summary."
Constraints:

Length: Target {target_token_range} tokens.
Preserve: Core conflict, protagonist's internal and external arcs,
emotional peaks, and moral/thematic undercurrents.
Remove: Detailed dialogue, minor characters with no plot impact,
specific scene descriptions, and fluff.
Specific Focus (The "Mission Keeper" Lens):

Highlight the Protagonist's Agency: How they drive the plot through
choices.
Identify Heroine's Journey markers: Internal awakening and reclaiming
of power.
Emphasize Moral Dilemmas: Points where the "Humanity Uplifting"
principles are tested.
Output Structure (Markdown Headers):

## Logline: A one-sentence summary.
## Thematic Core: The central moral question or "Uplift" theme.
## Character Profile: The protagonist's initial state vs. final state
(Arc).
## Structural Breakdown: Key beats (Inciting Incident, Midpoint,
Climax, Resolution) with emphasis on emotional transitions.
## Social Context: Any racial, cultural, or gender-specific dynamics
relevant to "Dignity & Inclusion."
""".strip()


def _build_reviewer_prompt(story_body: str, ctx: str) -> str:
    template = """Evaluate the submission for humanity-uplifting
value.

Return only this format:
Verdict: <Yes or No>
Score: <1 (out of 5) ... 5 (out of 5)>
Benefits:
- <2 to 5 concise bullets, not strictly required to reach 5 bullets>
Risks:
- <2 to 5 concise bullets, not strictly required to reach 5 bullets>
Overall rationale: <2 to 4 sentences>

Consistency rule:
- Score 1-2 => Verdict No
```

```
- Score 3-5 => Verdict Yes

Submission:
{story}

Additional Context:
{context}
"""
    return template.format(
        story=(story_body or "").strip(),
        context=(ctx or "").strip() if ctx else "None",
    )


def _summarize_script_for_reviewer(raw_script: str, filename: str,
target_token_range: str):
    global SUMMARY_GATE_REJECTED_KEY

    key = _current_google_api_key()
    if not key:
        return "", "", "Missing GOOGLE_API_KEY for summary gate"

    if SUMMARY_GATE_REJECTED_KEY and key == SUMMARY_GATE_REJECTED_KEY:
        return "", "", "GOOGLE_API_KEY previously rejected
(API_KEY_INVALID). Summary gate skipped until key changes."

    prompt = f"Here is the raw script treatment for '{filename}'.
Please summarize it:\n\n{raw_script}"

    candidates = []
    for model_name in [
        SCRIPT_SUMMARY_MODEL,
        SCRIPT_SUMMARY_MODEL_FALLBACK,
        "gemini-2.5-flash-lite",
        "gemini-1.5-flash",
    ]:
        if model_name and model_name not in candidates:
            candidates.append(model_name)

    client = genai.Client(api_key=key)
    last_err = None

    for model_name in candidates:
        try:
            full_prompt =
_summary_system_instruction(target_token_range) + "\n\n---\n\n" +
prompt
            response = client.models.generate_content(
                model=model_name,
                contents=full_prompt,
```

```
            )
            text = (getattr(response, "text", "") or "").strip()
            if text:
                return text, model_name, ""
        except Exception as e:
            last_err = e
            if _is_google_api_key_invalid_error(e):
                SUMMARY_GATE_REJECTED_KEY = key
                msg = "GOOGLE_API_KEY rejected by Gemini
(API_KEY_INVALID). Summary gate disabled until key is updated."
                print(f"[STEP 9 WARN] {msg}")
                return "", "", msg
            print(f"[STEP 9 WARN] Summary call failed on {model_name}:
{e}")

    return "", "", str(last_err) if last_err else "Unknown summary
error"




def _fit_story_to_reviewer_budget(story_text: str, context_text: str,
filename: str):
    original_tokens = _count_local_tokens(story_text)
    prompt_now = _build_reviewer_prompt(story_text, context_text)
    prompt_tokens_now = _count_local_tokens(prompt_now)

    result = {
        "story": story_text,
        "summary_applied": False,
        "summary_model": "",
        "summary_target": "",
        "original_tokens": original_tokens,
        "prompt_tokens": prompt_tokens_now,
        "error": "",
    }

    if prompt_tokens_now <= SCRIPT_REVIEWER_MAX_PROMPT_TOKENS and
original_tokens <= SCRIPT_SUMMARY_TRIGGER_TOKENS:
        return result

    current_story = story_text
    for target in SCRIPT_SUMMARY_TARGETS:
        summarized, model_used, err =
_summarize_script_for_reviewer(current_story, filename, target)
        if not summarized:
            result["error"] = err or result["error"]
            if err and ("API_KEY_INVALID" in err or "rejected by
Gemini" in err):
                break
```

```
            continue

        current_story = summarized
        prompt_now = _build_reviewer_prompt(current_story,
context_text)
        prompt_tokens_now = _count_local_tokens(prompt_now)

        result.update({
            "story": current_story,
            "summary_applied": True,
            "summary_model": model_used,
            "summary_target": target,
            "prompt_tokens": prompt_tokens_now,
            "error": "",
        })

        if prompt_tokens_now <= SCRIPT_REVIEWER_MAX_PROMPT_TOKENS:
            return result

    # Last-resort hard clip if still too long
    base_prompt_tokens =
_count_local_tokens(_build_reviewer_prompt("", context_text))
    max_story_tokens = max(256, SCRIPT_REVIEWER_MAX_PROMPT_TOKENS -
base_prompt_tokens - 64)
    clipped_story = _truncate_to_tokens(current_story,
max_story_tokens)
    final_prompt_tokens =
_count_local_tokens(_build_reviewer_prompt(clipped_story,
context_text))

    result.update({
        "story": clipped_story,
        "summary_applied": True,
        "prompt_tokens": final_prompt_tokens,
    })
    return result


def _print_raw_model_output(raw_text: str, tag: str = ""):
    header = f"[MODEL RAW OUTPUT] {tag}".strip()
    print("\n" + "=" * 100)
    print(header)
    print("=" * 100)
    print(raw_text if (raw_text or "").strip() else "(empty)")
    print("=" * 100 + "\n")


def _generate_reviewer_output(
    prompt: str,
    *,
```

```
    max_new_tokens: int,
    min_new_tokens: int,
    temperature: float,
    top_p: float,
    do_sample: bool,
):
    inputs = tok(prompt, return_tensors="pt").to(device)

    gen_kwargs = {
        "max_new_tokens": max_new_tokens,
        "min_new_tokens": min_new_tokens,
        "temperature": temperature,
        "top_p": top_p,
        "do_sample": do_sample,
        "repetition_penalty": 1.12,
        "no_repeat_ngram_size": 5,
    }

    eos_id = getattr(tok, "eos_token_id", None)
    if eos_id is not None:
        gen_kwargs["pad_token_id"] = eos_id

    outputs = mdl.generate(**inputs, **gen_kwargs)
    prompt_len = inputs["input_ids"].shape[1]
    raw_text = tok.decode(outputs[0][prompt_len:],
skip_special_tokens=True).strip()
    _print_raw_model_output(raw_text, "script-reviewer")
    cleaned = _clean_model_output(raw_text)
    return cleaned, raw_text


def evaluate_story(story, context, uploaded_file):
    with gen_lock:
        if (not story or story.strip() == "") and not uploaded_file:
            return "Please enter a story or upload a file.", 1,
history

        file_text = ""
        if uploaded_file is not None:
            path = uploaded_file.name
            ext = os.path.splitext(path)[1].lower()
            try:
                if ext == ".txt":
                    with open(path, "r", encoding="utf-8",
errors="ignore") as f:
                        file_text = f.read()
                elif ext in (".docx", ".doc"):
                    import docx
                    doc = docx.Document(path)
                    file_text = "\n".join([p.text for p in
```

```
doc.paragraphs])
                elif ext == ".pdf":
                    import PyPDF2
                    with open(path, "rb") as f:
                        reader = PyPDF2.PdfReader(f)
                        file_text = "\n".join([page.extract_text() or
"" for page in reader.pages])
                elif ext.startswith('.g'):
                    file_text = read_google_workspace_file(path)
                else:
                    file_text = "(File type not supported)"
            except Exception as e:
                file_text = f"(Error reading file: {str(e)})"

        final_story = (story or "") + "\n\n" + file_text
        if final_story.strip() == "":
            return "Input is empty.", 1, history

        prompt = _build_reviewer_prompt(
            final_story.strip(),
            context.strip() if context else "None",
        )

        cleaned_text, raw_text = _generate_reviewer_output(
            prompt,
            max_new_tokens=220,
            min_new_tokens=0,
            temperature=0.0,
            top_p=1.0,
            do_sample=False,
        )

        parsed = format_output(cleaned_text)
        quality = _parsed_quality(parsed)

        # Auto-retry disabled for stability (kept code for quick re-
enable).
        if False and (quality < 7 or len(cleaned_text) < 180):
            repair_prompt = prompt + "\n\nIMPORTANT: Produce exactly
one [BEGIN_REVIEW]...[END_REVIEW] block with no extra text. Keep
overall rationale at 2-4 sentences."
            retry_cleaned, retry_raw = _generate_reviewer_output(
                repair_prompt,
                max_new_tokens=560,
                min_new_tokens=180,
                temperature=0.0,
                top_p=1.0,
                do_sample=False,
            )
            retry_parsed = format_output(retry_cleaned)
```

```
            retry_quality = _parsed_quality(retry_parsed)

            if retry_quality >= quality or len(retry_cleaned) >
len(cleaned_text):
                cleaned_text = retry_cleaned
                raw_text = retry_raw
                parsed = retry_parsed
                quality = retry_quality

        final_score = _parse_score_to_int(parsed.get("score", ""),
cleaned_text)

        score_based_verdict = "Yes" if final_score >= 3 else "No"
        parsed_verdict = _normalize_verdict(parsed.get("verdict", ""))
        if parsed_verdict and parsed_verdict != score_based_verdict:
            print(f"[STEP 9 WARN] Verdict/Score mismatch from model:
verdict={parsed_verdict}, score={final_score}. Using score-based
verdict.")
        verdict_text = score_based_verdict
        score_text = f"{final_score} (out of 5)"
        benefits_text = (parsed.get("benefits") or "").strip() or "-
Not clearly provided by the model."
        risks_text = (parsed.get("risks") or "").strip() or "- Not
clearly provided by the model."
        overall_text = (parsed.get("overall") or "").strip() or
cleaned_text

        history.append({
            "story": (final_story[:200] + "...") if len(final_story) >
200 else final_story,
            "score": final_score,
            "verdict": verdict_text,
            "output": (cleaned_text[:300] + "...") if
len(cleaned_text) > 300 else cleaned_text,
        })

        display_text = f"""
### Verdict
{verdict_text}

### Score
{score_text}

### Benefits
{benefits_text}

### Risks
{risks_text}

### Overall Rationale
```

```
{overall_text}
""".strip()

        # Show raw model text only when parsed structure is weak.
        if quality < 8:
            debug_raw = (parsed.get("raw") or cleaned_text)[:2400]
            display_text += f"\n\n### Raw Model Output (debug)\
n```text\n{debug_raw}\n```"

        return display_text, final_score, history




print("[STEP 9.1] Evaluator ready.")
```

```
[STEP 9] Defining evaluator...
[STEP 9.1] Evaluator ready.
```

Section D - Impact / RAG Backend

Impact backend components, retrieval, generation, and indexing helpers.

STEP 13: Impact Agent Backend (Copied from Impact_RAG_v4)

This section is copied from `Impact_RAG_v4.ipynb` with minimal modifications. It defines the RAG-backed impact analysis path used by the unified dispatcher.

```
# === Choose your generator backend ===
# "local" uses a small open-source chat model via Hugging Face
Transformers.
# "openai" uses OpenAI's API (set OPENAI_API_KEY).
# "gemini" uses Google Generative AI (set GOOGLE_API_KEY).
GEN_BACKEND = "gemini"  # options: "local", "openai", "gemini"

# Local model config:
LOCAL_MODEL = "TinyLlama/TinyLlama-1.1B-Chat-v1.0"  # light model for
CPU/GPU demos

# Retrieval config
EMBEDDING_MODEL = "sentence-transformers/all-MiniLM-L6-v2"
CHUNK_SIZE = 800
CHUNK_OVERLAP = 120
TOP_K = 4

# Prompt template
SYSTEM_PROMPT = (
    "You are a helpful assistant. Use the provided context to answer
```

```
the user's question.\n"
    "If the answer is not in the context, say you don't know.\n"
)

ANSWER_TEMPLATE = """[System]
{system}

[Context]
{context}

[User Question]
{question}

[Instructions]
- Cite the most relevant chunks briefly (e.g., 'From chunk 2').
- If unsure, say 'I don't know from the provided docs.'
- Keep answers concise and factual.
"""

OPENAI_API_KEY = globals().get("OPENAI_API_KEY",
os.getenv("OPENAI_API_KEY", ""))
GOOGLE_API_KEY = globals().get("GOOGLE_API_KEY",
os.getenv("GOOGLE_API_KEY", ""))

print(f"OPENAI_API_KEY: {OPENAI_API_KEY[:4]}{'*' *
(len(OPENAI_API_KEY) - 4) if OPENAI_API_KEY else ''}")
print(f"GOOGLE_API_KEY: {GOOGLE_API_KEY[:4]}{'*' *
(len(GOOGLE_API_KEY) - 4) if GOOGLE_API_KEY else ''}")
```

```
OPENAI_API_KEY:
GOOGLE_API_KEY: AIza**********************************
```

```
# Copied runtime imports from Impact_RAG_v4
import os, torch, faiss
from sentence_transformers import SentenceTransformer
from langchain_text_splitters import RecursiveCharacterTextSplitter

print("Impact backend runtime | PyTorch:", torch.__version__, "|
CUDA:", torch.cuda.is_available())
device = "cuda" if torch.cuda.is_available() else "cpu"
```

```
Impact backend runtime | PyTorch: 2.10.0+cu128 | CUDA: True
```

```
import os
from typing import List, Dict
from pypdf import PdfReader
from docx import Document  # ADDED THIS LINE

def load_texts_from_paths(paths: List[str]) -> List[Dict]:
    """
    Load text from various file formats: PDF, DOCX, DOC, TXT, MD
```

```
    """
    docs = []
    for p in paths:
        text = ""

        # PDF files
        if p.lower().endswith(".pdf"):
            try:
                reader = PdfReader(p)
                for page in reader.pages:
                    text += page.extract_text() or ""
                print(f"✓ Loaded PDF: {os.path.basename(p)}
({len(text)} chars)")
            except Exception as e:
                print(f"[WARN] Failed to parse PDF {p}: {e}")
                continue

        # DOCX files (Word documents) # NEW
        elif p.lower().endswith(".docx"):
            try:
                doc = Document(p)
                # Extract text from all paragraphs
                text = "\n".join([paragraph.text for paragraph in
doc.paragraphs])
                # Also extract text from tables
                for table in doc.tables:
                    for row in table.rows:
                        for cell in row.cells:
                            text += "\n" + cell.text
                print(f"✓ Loaded DOCX: {os.path.basename(p)}
({len(text)} chars)")
            except Exception as e:
                print(f"[WARN] Failed to parse DOCX {p}: {e}")
                continue

        # DOC files (legacy Word format) # NEW
        elif p.lower().endswith(".doc"):
            try:
                doc = Document(p)
                text = "\n".join([paragraph.text for paragraph in
doc.paragraphs])
                print(f"✓ Loaded DOC: {os.path.basename(p)}
({len(text)} chars)")
            except Exception as e:
                print(f"[WARN] Failed to parse DOC {p}: {e}")
                print("     Note: Legacy .doc files may require
conversion to .docx")
                continue

        # TXT and MD files
```

```
        elif p.lower().endswith((".txt", ".md")):
            try:
                with open(p, "r", encoding="utf-8", errors="ignore")
as f:
                    text = f.read()
                print(f"✓ Loaded TXT/MD: {os.path.basename(p)}
({len(text)} chars)")
            except Exception as e:
                print(f"[WARN] Failed to read text file {p}: {e}")
                continue

        else:
            print(f"[SKIP] Unsupported file type: {p}")
            continue

        if text.strip():  # Only add if we got some text
            docs.append({"path": p, "text": text})
        else:
            print(f"[WARN] No text extracted from {p}")

    return docs

splitter = RecursiveCharacterTextSplitter(
    chunk_size=CHUNK_SIZE, chunk_overlap=CHUNK_OVERLAP,
length_function=len
)

def chunk_docs(docs: List[Dict]) -> List[Dict]:
    chunks = []
    for d in docs:
        for i, ch in enumerate(splitter.split_text(d["text"])):
            chunks.append({"source": d["path"], "chunk_id": i, "text":
ch})
    return chunks

class RAGIndex:
    def __init__(self, embedding_model_name: str):
        self.model = SentenceTransformer(embedding_model_name,
device=device)
        self.index = None
        self.chunks: List[Dict] = []

    def build(self, chunks: List[Dict]):
        self.chunks = chunks
        embs = self.model.encode([c["text"] for c in chunks],
convert_to_numpy=True, show_progress_bar=True)
        dim = embs.shape[1]
        index = faiss.IndexFlatIP(dim)
        faiss.normalize_L2(embs)
        index.add(embs)
```

```
        self.index = index
        print(f"Built index with {len(chunks)} chunks.")

    def search(self, query: str, k: int = 4):
        if self.index is None or not self.chunks:
            return []
        q = self.model.encode([query], convert_to_numpy=True)
        faiss.normalize_L2(q)
        scores, idxs = self.index.search(q, k)
        results = []
        for score, idx in zip(scores[0], idxs[0]):
            if idx == -1: continue
            results.append((float(score), self.chunks[idx]))
        return results

rag = RAGIndex(EMBEDDING_MODEL)
```


```
{"model_id":"9a75ec4026404e3bb6c0f9f1c663ca6d","version_major":2,"version_minor":0}

{"model_id":"3ada7a9a44cb42e3828c807406330401","version_major":2,"version_minor":0}

{"model_id":"f8aafd367117489194d8fc805bea73a0","version_major":2,"version_minor":0}

{"model_id":"2a6278456bbd47899118e4749a570839","version_major":2,"version_minor":0}

{"model_id":"a2253040dc064f0593ebf15e618d0324","version_major":2,"version_minor":0}

{"model_id":"20a0c3800d004d40a1f46a050b8e3c1d","version_major":2,"version_minor":0}

{"model_id":"7870e07c3c464ebeb227efa0004a7948","version_major":2,"version_minor":0}

{"model_id":"d3e83c14538045c7a2430f41af661524","version_major":2,"version_minor":0}

{"model_id":"83f25fccaeb14821acb65454ea2b9def","version_major":2,"version_minor":0}

{"model_id":"8af8a8f581024425a29667e6e99952d7","version_major":2,"version_minor":0}

{"model_id":"16605674c52d42fdb60c3eb2e318624c","version_major":2,"version_minor":0}

{"model_id":"efdceb071a2145c09ee1bc6dd8c58915","version_major":2,"version_minor":0}
```

```
def render_context(snippets):
    lines = []
    for rank, (score, ch) in enumerate(snippets, start=1):
        header = f"[Chunk {rank}] (score={score:.3f})
source={os.path.basename(ch['source'])} id={ch['chunk_id']}"
        lines.append(header + "\n" + ch["text"])
    return "\n\n".join(lines)

def build_prompt(question, context_blocks):
    return ANSWER_TEMPLATE.format(
        system=SYSTEM_PROMPT.strip(),
        context=context_blocks.strip(),
        question=question.strip()
    )

from transformers import AutoModelForCausalLM, AutoTokenizer, pipeline
import torch

_local_pipe = None

def get_local_pipe():
    global _local_pipe
    if _local_pipe is None:
        tok = AutoTokenizer.from_pretrained(LOCAL_MODEL,
use_fast=True)
        model = AutoModelForCausalLM.from_pretrained(
            LOCAL_MODEL,
            device_map="auto" if torch.cuda.is_available() else None,
            torch_dtype=torch.float16 if torch.cuda.is_available()
else torch.float32,
            low_cpu_mem_usage=True
        )
        _local_pipe = pipeline(
            "text-generation",
            model=model,
            tokenizer=tok,
            device=0 if torch.cuda.is_available() else -1,
            max_new_tokens=384,
            do_sample=True,
            temperature=0.3,
            top_p=0.9,
            repetition_penalty=1.05
        )
    return _local_pipe

def generate_local(prompt: str) -> str:
    p = get_local_pipe()
    out = p(prompt, pad_token_id=p.tokenizer.eos_token_id)[0]
["generated_text"]
    return out[len(prompt):].strip()
```

```python
def generate_openai(prompt: str) -> str:
    if not OPENAI_API_KEY:
        return "OPENAI_API_KEY not set. Switch GEN_BACKEND to 'local'
or set your key."
    try:
        from openai import OpenAI
        client = OpenAI(api_key=OPENAI_API_KEY)
        r = client.chat.completions.create(
            model="gpt-4o-mini",
            messages=[{"role":"system","content":SYSTEM_PROMPT},
                      {"role":"user","content":prompt}],
            temperature=0.2,
            max_tokens=400
        )
        return r.choices[0].message.content
    except Exception as e:
        return f"[OpenAI error] {e}"

def _strip_local_proxy_env():
    # VS Code Colab plugin may inject localhost proxy envs that
timeout.
    keys = [
        "HTTP_PROXY", "HTTPS_PROXY", "ALL_PROXY",
        "http_proxy", "https_proxy", "all_proxy",
    ]
    for k in keys:
        v = os.environ.get(k, "")
        if "localhost:" in v or "127.0.0.1:" in v:
            os.environ.pop(k, None)


def _extract_gemini_text(resp_json: dict) -> str:
    cands = resp_json.get("candidates", [])
    if not cands:
        return ""
    parts = cands[0].get("content", {}).get("parts", [])
    texts = [p.get("text", "") for p in parts if isinstance(p, dict)]
    return "\n".join([t for t in texts if t]).strip()


def _generate_gemini_via_sdk(prompt: str, model_name: str = "gemini-
2.5-flash-lite", timeout_sec: int = 35) -> str:
    from google import genai

    client = genai.Client(api_key=GOOGLE_API_KEY)
    r = client.models.generate_content(model=model_name,
contents=prompt)
    return (getattr(r, "text", "") or "").strip()
```

```python
def _generate_gemini_via_rest(prompt: str, model_name: str = "gemini-
2.5-flash-lite", timeout_sec: int = 35) -> str:
    import requests

    url =
f"https://generativelanguage.googleapis.com/v1beta/models/{model_name}
:generateContent"
    payload = {"contents": [{"parts": [{"text": prompt}]}]}

    # Explicitly disable proxies for this request.
    r = requests.post(
        url,
        params={"key": GOOGLE_API_KEY},
        json=payload,
        timeout=timeout_sec,
        proxies={"http": "", "https": ""},
    )
    r.raise_for_status()
    data = r.json()
    text = _extract_gemini_text(data)
    if text:
        return text
    return f"[Gemini REST response without text] {str(data)[:400]}"


def generate_gemini(prompt: str) -> str:
    if not GOOGLE_API_KEY:
        return "GOOGLE_API_KEY not set. Switch GEN_BACKEND to 'local'
or set your key."

    _strip_local_proxy_env()

    try:
        return _generate_gemini_via_sdk(prompt)
    except Exception as sdk_err:
        try:
            return _generate_gemini_via_rest(prompt)
        except Exception as rest_err:
            return f"[Gemini error] sdk={sdk_err} | rest={rest_err}"

def answer_question(question: str, top_k: int = TOP_K):
    hits = rag.search(question, k=top_k)
    context = render_context(hits)
    prompt = build_prompt(question, context)

    if GEN_BACKEND == "local":
        answer = generate_local(prompt)
    elif GEN_BACKEND == "openai":
        answer = generate_openai(prompt)
    elif GEN_BACKEND == "gemini":
```

```
        answer = generate_gemini(prompt)
    else:
        answer = f"Unknown backend: {GEN_BACKEND}"

    return {"question": question, "answer": answer, "top_chunks":
hits}

print("RAG ready. After indexing, call: answer_question('Your
query')")
```

```
RAG ready. After indexing, call: answer_question('Your query')
```

```
import os
from typing import List

SUPPORTED_INDEX_EXTS = (".pdf", ".txt", ".md", ".docx", ".doc")


def _collect_supported_files(scan_dir: str) -> List[str]:
    paths = []
    if not scan_dir or not os.path.isdir(scan_dir):
        return paths

    for root, _, files in os.walk(scan_dir):
        for name in files:
            if name.lower().endswith(SUPPORTED_INDEX_EXTS):
                paths.append(os.path.join(root, name))
    return sorted(paths)


def _upload_files_widget() -> List[str]:
    """Works only in active Chrome Colab browser sessions."""
    try:
        from google.colab import files
    except Exception as e:
        print(f"Colab upload widget unavailable: {e}")
        return []

    try:
        print("Select PDFs/TXT/MD/DOCX/DOC files...")
        uploaded = files.upload()
    except Exception as e:
        print(f"Upload widget failed: {e}")
        return []

    paths = []
    target_dir = "/content" if os.path.isdir("/content") else
os.getcwd()
    for name, data in uploaded.items():
        out_path = os.path.join(target_dir, name)
        with open(out_path, "wb") as f:
```

```
            f.write(data)
        paths.append(out_path)
    return paths


def resolve_index_paths(
    manual_paths: List[str] = None,
    scan_dir: str = "",
    use_widget_upload: bool = False,
) -> List[str]:
    # 1) Highest priority: explicit manual paths
    manual_paths = manual_paths or []
    existing = [p for p in manual_paths if isinstance(p, str) and
os.path.isfile(p)]
    if existing:
        return existing

    # 2) Optional browser widget upload (Chrome Colab only)
    if use_widget_upload:
        widget_paths = _upload_files_widget()
        if widget_paths:
            return widget_paths
        print("Widget upload returned no files. Falling back to
directory scan...")

    # 3) Directory scan fallback
    scanned = _collect_supported_files(scan_dir)
    if scanned:
        print(f"Found {len(scanned)} supported files in: {scan_dir}")
    return scanned


def build_index_from_paths(paths: List[str]):
    if not paths:
        raise ValueError("No input files found. Set MANUAL_PATHS or
SCAN_DIR first.")

    docs = load_texts_from_paths(paths)
    chunks = chunk_docs(docs)
    rag.build(chunks)
    print(f"Indexed {len(chunks)} chunks from {len(docs)} files.")
```

Optional: Build / Rebuild RAG Index

Use this only when you want to index files for Impact Agent retrieval.

```
# Optional index build command (manual)
# Example:
# paths =
```

```
resolve_index_paths(manual_paths=["/content/drive/MyDrive/your_script.
pdf"], scan_dir="/content/drive/MyDrive", use_widget_upload=False)
# build_index_from_paths(paths)
# print("Chunks indexed:", len(rag.chunks))
```

Section E - Unified Gradio UI

Integrated user-facing interface and launch path.

STEP 14: Unified UI (V4 Modern Console)

- Modernized UI system with a cleaner visual hierarchy and status rail.
- Keeps orchestrator dispatch and model behavior unchanged.
- Adds live execution stage indicators (Agent / Route Mode / Stage).
- Adds conversation memory injection and local chat history persistence.

Note: This cell defines `demo` but does not launch it.

```
import os
import json
import gradio as gr
import inspect
import html
from types import SimpleNamespace
from pypdf import PdfReader
from docx import Document


def _extract_uploaded_path(uploaded_file) -> str:
    if uploaded_file is None:
        return ""
    if isinstance(uploaded_file, str):
        return uploaded_file
    if isinstance(uploaded_file, list) and uploaded_file:
        return _extract_uploaded_path(uploaded_file[0])
    if isinstance(uploaded_file, dict):
        return uploaded_file.get("name", "") or
uploaded_file.get("path", "")
    if hasattr(uploaded_file, "name"):
        return uploaded_file.name
    return ""


def read_file_to_string(file_path: str) -> str:
    """Reads a TXT, PDF, DOCX, or DOC file into a single text
string."""
    if not file_path:
        return ""
```

```
    text = ""
    file_ext = os.path.splitext(file_path)[1].lower()

    try:
        if file_ext == ".pdf":
            reader = PdfReader(file_path)
            for page in reader.pages:
                text += page.extract_text() or ""

        elif file_ext == ".docx":
            doc = Document(file_path)
            text = "\n".join([p.text for p in doc.paragraphs])
            for table in doc.tables:
                for row in table.rows:
                    for cell in row.cells:
                        text += "\n" + cell.text

        elif file_ext == ".doc":
            try:
                doc = Document(file_path)
                text = "\n".join([p.text for p in doc.paragraphs])
            except Exception:
                return "ERROR: Legacy .doc detected. Please resave
as .docx and upload again."

        elif file_ext in [".txt", ".md"]:
            with open(file_path, "r", encoding="utf-8",
errors="replace") as f:
                text = f.read()

        else:
            return f"ERROR: Unsupported file type: {file_ext}"

    except Exception as e:
        return f"ERROR: Could not read file ({e})"

    return text


def on_upload(uploaded_file):
    file_path = _extract_uploaded_path(uploaded_file)
    if not file_path:
        return "", "No file attached.", ""

    file_text = read_file_to_string(file_path)
    if file_text.startswith("ERROR:"):
        return "", file_text, ""

    filename = os.path.basename(file_path)
    return file_text, f"Attached: {filename}", file_path
```

```python
def clear_uploaded_state():
    return "", "No file attached.", ""


def _combine_user_input(story_text: str, uploaded_text: str) -> str:
    story = (story_text or "").strip()
    file_body = (uploaded_text or "").strip()

    if story and file_body:
        return f"{story}\n\n[Uploaded File Content]\n{file_body}"
    return story or file_body


def _format_user_turn(story_text: str, uploaded_text: str) -> str:
    story = (story_text or "").strip()
    has_file = bool((uploaded_text or "").strip())

    if story and has_file:
        return story + "\n[Attachment included]"
    if story:
        return story
    if has_file:
        return "[Attachment only submission]"
    return ""


def _status_html(agent: str = "", route_mode: str = "", stage: str =
"Idle") -> str:
    agent_value = html.escape((agent or "--").strip() or "--")
    mode_value = html.escape((route_mode or "--").strip() or "--")
    stage_value = html.escape((stage or "Idle").strip() or "Idle")

    return (
        '<div class="status-strip">'
        f'<div class="status-pill"><span class="k">Agent</span><span
class="v">{agent_value}</span></div>'
        f'<div class="status-pill"><span class="k">Route</span><span
class="v">{mode_value}</span></div>'
        f'<div class="status-pill"><span class="k">Stage</span><span
class="v">{stage_value}</span></div>'
        '</div>'
    )


MEMORY_MAX_TURNS = 4
MEMORY_MAX_CHARS = 3200
MEMORY_MAX_MSG_CHARS = 480
TRANSIENT_ASSISTANT_PREFIXES = (
```

```
    "Analyzing your submission...",
    "Routing request to the best agent...",
    "Routed to `",
    "Running Impact Agent",
    "Preparing Script Reviewer input...",
    "Input length check:",
    "Long script detected.",
    "Summary gate complete",
    "Summary gate not needed.",
    "Running Script Reviewer model inference...",
)


def _chat_history_path() -> str:
    raw_path = (os.getenv("IMPACT_CHAT_HISTORY_PATH", "") or
"").strip()
    if not raw_path:
        raw_path = ".impactstudio_chat_history.json"
    return os.path.abspath(raw_path)


def _normalize_history(history):
    normalized = []
    for item in history or []:
        if not isinstance(item, dict):
            continue
        role = item.get("role")
        content = item.get("content")
        if role in ("user", "assistant") and isinstance(content, str):
            normalized.append({"role": role, "content": content})
    return normalized


def _load_persisted_chat_history():
    path = _chat_history_path()
    if not os.path.exists(path):
        return []
    try:
        with open(path, "r", encoding="utf-8") as f:
            data = json.load(f)
        return _normalize_history(data if isinstance(data, list) else
[])
    except Exception as e:
        print(f"[STEP 14 WARN] Could not load persisted chat history:
{e}")
        return []


def _persist_chat_history(history):
    path = _chat_history_path()
```

```
    tmp_path = path + ".tmp"
    payload = _normalize_history(history)
    try:
        parent = os.path.dirname(path)
        if parent:
            os.makedirs(parent, exist_ok=True)
        with open(tmp_path, "w", encoding="utf-8") as f:
            json.dump(payload, f, ensure_ascii=False, indent=2)
        os.replace(tmp_path, path)
    except Exception as e:
        print(f"[STEP 14 WARN] Could not persist chat history: {e}")


def _is_transient_assistant_text(text: str) -> bool:
    clean = (text or "").strip()
    return any(clean.startswith(prefix) for prefix in
TRANSIENT_ASSISTANT_PREFIXES)


def _truncate_for_memory(text: str, max_chars: int =
MEMORY_MAX_MSG_CHARS) -> str:
    clean = (text or "").strip()
    if len(clean) <= max_chars:
        return clean
    return clean[: max_chars - 3].rstrip() + "..."


def _strip_response_meta_for_memory(text: str) -> str:
    drop_prefixes = (
        "**Routed Agent:**",
        "**Route Mode:**",
        "**Summary Gate:**",
        "**Summary Note:**",
    )
    lines = []
    for line in (text or "").splitlines():
        if any(line.strip().startswith(prefix) for prefix in
drop_prefixes):
            continue
        lines.append(line)
    return _truncate_for_memory("\n".join(lines).strip())


def _history_before_current_turn(history):
    h = list(history or [])
    if len(h) >= 2 and h[-2].get("role") == "user" and h[-
1].get("role") == "assistant":
        if _is_transient_assistant_text(h[-1].get("content", "")):
            return h[:-2]
    return h
```

```python
def _build_memory_context(chat_history, max_turns: int =
MEMORY_MAX_TURNS) -> str:
    flat = []
    for msg in _normalize_history(chat_history):
        role = msg.get("role")
        content = (msg.get("content") or "").strip()
        if not content:
            continue
        if role == "assistant":
            if _is_transient_assistant_text(content):
                continue
            content = _strip_response_meta_for_memory(content)
        else:
            content = _truncate_for_memory(content)
        if not content:
            continue
        speaker = "User" if role == "user" else "Assistant"
        flat.append(f"{speaker}: {content}")

    if not flat:
        return ""

    recent = flat[-(max_turns * 2):]
    memory = "\n\n".join(recent).strip()
    if len(memory) > MEMORY_MAX_CHARS:
        memory = "...\n" + memory[-(MEMORY_MAX_CHARS - 4):]
    return memory


def _build_route_prompt(route_basis: str, memory_context: str) -> str:
    memory = (memory_context or "").strip()
    if not memory:
        return route_basis
    return (
        f"Conversation context (recent turns):\n{memory}\n\n"
        f"Current user request:\n{route_basis}"
    )


def _augment_mission_with_memory(mission_text: str, memory_context:
str) -> str:
    mission = (mission_text or "").strip()
    memory = (memory_context or "").strip()
    if not memory:
        return mission
    memory_block = f"Conversation memory (recent turns):\n{memory}"
    return f"{mission}\n\n{memory_block}" if mission else memory_block
```

```
def _yield_ui_state(history, payload, status_html, persist: bool =
True):
    if persist:
        _persist_chat_history(history)
    return history, history, payload, status_html


def _gradio_story_eval(story_text: str, mission: str = "",
memory_context: str = ""):
    """Impact agent path (copied from v4)."""
    if not story_text.strip():
        return "Please enter text or upload a file before submitting."

    base_prompt = """
    You are an editorial advisor focused on community engagement and
ethical publication. Read the following piece of writing
    and identify the communities, audiences, or individuals who may be
directly affected by its themes. Then provide specific,
    actionable suggestions for how the author can responsibly and
meaningfully reach out to or support those communities.

    Your feedback must:
    - Be concrete (exact additions, placements, wording ideas, or
resources).
    - Explain why each suggestion is appropriate for the content.
    - Address ethical considerations such as harm reduction,
accessibility, and care for vulnerable readers when relevant.

    Avoid generic advice like "be sensitive" or "raise awareness."

    When users share a script, concept, or idea, your job is to:
    1. Analyze the submission's tone, themes, and emotional depth.
    2. Determine the topics discussed in the submission.
    3. Provide clear reasoning.
    4. Provide specific and actionable outreach/support suggestions.
    5. Integrate any mission context naturally.

    Respond in this format:

    Topics identified:
    (List topics found in the submission. For each topic include one
quote and one actionable recommendation.)
    """

    memory_block = ""
    if (memory_context or "").strip():
        memory_block = f"\n\nConversation Memory (recent turns):\
n{memory_context.strip()}"
```

```
    if mission and mission.strip():
        user_prompt = (
            f"{base_prompt}\n\nCurrent Studio Mission:\
n{mission.strip()}{memory_block}\n\nUser Submission:\
n{story_text.strip()}"
        )
    else:
        user_prompt = f"{base_prompt}{memory_block}\n\nUser
Submission:\n{story_text.strip()}"

    context_text = ""
    if "rag" in globals() and getattr(rag, "chunks", []):
        hits = rag.search(story_text, k=TOP_K)
        context_text = render_context(hits)
    else:
        hits = []

    full_prompt = ANSWER_TEMPLATE.format(
        system=SYSTEM_PROMPT,
        context=context_text or "[No extra context]",
        question=user_prompt,
    )

    out = answer_question(full_prompt)
    answer = out.get("answer", "")

    cites = []
    if context_text:
        for rank, (_, ch) in enumerate(out.get("top_chunks", []),
start=1):
            cites.append(f"Chunk {rank} -
{os.path.basename(ch['source'])}#{ch['chunk_id']}")
    suffix = ("\n\n---\nSources: " + "; ".join(cites)) if cites else
""

    return (answer or "[Empty answer]") + suffix


def _to_uploaded_file_obj(path: str):
    if not path:
        return None
    return SimpleNamespace(name=path)


def _dispatch_request(story_text: str, mission_text: str,
uploaded_text: str, uploaded_path: str, chat_history=None):
    if "safe_route" not in globals():
        return "error", "missing_safe_route", "Routing module not
initialized. Run orchestrator cells first."
```

```
    merged_submission = _combine_user_input(story_text, uploaded_text)
    route_basis = (story_text or "").strip() or
merged_submission[:1200]
    if not route_basis:
        return "error", "empty_input", "Please enter text or upload a
file before submitting."

    memory_context = _build_memory_context(chat_history)
    effective_mission = _augment_mission_with_memory(mission_text or
"", memory_context)
    routed_agent, route_mode =
safe_route(_build_route_prompt(route_basis, memory_context))

    if routed_agent == orchestrator.IMPACT_AGENT:
        answer_body = _gradio_story_eval(merged_submission,
effective_mission, memory_context)
    else:
        file_obj = _to_uploaded_file_obj(uploaded_path)
        script_input = (story_text or "").strip()
        answer_body, _, _ = evaluate_story(script_input,
effective_mission, file_obj)

    final_answer = f"**Routed Agent:** `{routed_agent}`\n\n**Route
Mode:** `{route_mode}`\n\n{answer_body}"
    return routed_agent, route_mode, final_answer


def stage_user_message(story_text: str, mission_text: str,
uploaded_text: str, uploaded_path: str, chat_history):
    merged_submission = _combine_user_input(story_text, uploaded_text)
    if not merged_submission.strip():
        history = list(chat_history or [])
        return history, "", history, {}, _status_html(stage="Idle")

    history = list(chat_history or [])
    history.append({"role": "user", "content":
_format_user_turn(story_text, uploaded_text)})
    history.append({"role": "assistant", "content": "Analyzing your
submission..."})

    payload = {
        "story_text": story_text or "",
        "mission_text": mission_text or "",
        "uploaded_text": uploaded_text or "",
        "uploaded_path": uploaded_path or "",
    }
    _persist_chat_history(history)
    return history, "", history, payload, _status_html(stage="Queued")
```

```
def _set_assistant_status(history, text):
    history = list(history or [])
    msg = {"role": "assistant", "content": text}
    if history and history[-1].get("role") == "assistant":
        history[-1] = msg
    else:
        history.append(msg)
    return history


def resolve_assistant_message(chat_history, pending_payload):
    history = list(chat_history or [])
    payload = pending_payload or {}

    if not payload:
        yield _yield_ui_state(history, {}, _status_html(stage="Idle"),
persist=False)
        return

    story_text = payload.get("story_text", "")
    mission_text = payload.get("mission_text", "")
    uploaded_text = payload.get("uploaded_text", "")
    uploaded_path = payload.get("uploaded_path", "")
    prior_history = _history_before_current_turn(history)
    memory_context = _build_memory_context(prior_history)
    effective_mission_text = _augment_mission_with_memory(mission_text
or "", memory_context)

    merged_submission = _combine_user_input(story_text, uploaded_text)
    route_basis = (story_text or "").strip() or
merged_submission[:1200]

    if not route_basis:
        history = _set_assistant_status(history, "Please enter text or
upload a file before submitting.")
        yield _yield_ui_state(history, {}, _status_html(stage="Input
Error"))
        return

    history = _set_assistant_status(history, "Routing request to the
best agent...")
    yield _yield_ui_state(history, payload,
_status_html(stage="Routing"))

    if "safe_route" not in globals():
        history = _set_assistant_status(history, "Routing module not
initialized. Run orchestrator cells first.")
        yield _yield_ui_state(history, {}, _status_html(stage="Routing
Error"))
        return
```

```
    routed_agent, route_mode =
safe_route(_build_route_prompt(route_basis, memory_context))
    history = _set_assistant_status(history, f"Routed to
`{routed_agent}` via `{route_mode}`.")
    yield _yield_ui_state(history, payload, _status_html(routed_agent,
route_mode, stage="Routed"))

    if routed_agent == orchestrator.IMPACT_AGENT:
        history = _set_assistant_status(history, "Running Impact Agent
(RAG retrieval + generation)...")
        yield _yield_ui_state(history, payload,
_status_html(routed_agent, route_mode, stage="Impact Agent Running"))

        answer_body = _gradio_story_eval(merged_submission,
effective_mission_text, memory_context)
        final_answer = f"**Routed Agent:** `{routed_agent}`\n\n**Route
Mode:** `{route_mode}`\n\n{answer_body}"

        history = _set_assistant_status(history, final_answer)
        yield _yield_ui_state(history, {}, _status_html(routed_agent,
route_mode, stage="Completed"))
        return

    filename_hint = os.path.basename(uploaded_path) if uploaded_path
else "submission"

    history = _set_assistant_status(history, "Preparing Script
Reviewer input...")
    yield _yield_ui_state(history, payload, _status_html(routed_agent,
route_mode, stage="Script Reviewer Prep"))

    initial_prompt_tokens =
_count_local_tokens(_build_reviewer_prompt(merged_submission,
effective_mission_text))
    history = _set_assistant_status(history, f"Input length check:
~{initial_prompt_tokens} tokens.")
    yield _yield_ui_state(history, payload, _status_html(routed_agent,
route_mode, stage="Token Budget Check"))

    needs_summary = (
        initial_prompt_tokens > SCRIPT_REVIEWER_MAX_PROMPT_TOKENS
        or _count_local_tokens(merged_submission) >
SCRIPT_SUMMARY_TRIGGER_TOKENS
    )
    if needs_summary:
        history = _set_assistant_status(history, "Long script
detected. Running Gemini summary gate...")
        yield _yield_ui_state(history, payload,
_status_html(routed_agent, route_mode, stage="Summary Gate"))
```

```
    fit_result = _fit_story_to_reviewer_budget(merged_submission,
effective_mission_text, filename_hint)

    if fit_result.get("summary_applied"):
        history = _set_assistant_status(
            history,
            f"Summary gate complete via
`{fit_result.get('summary_model')}` "
            f"(target {fit_result.get('summary_target')},
prompt≈{fit_result.get('prompt_tokens')} tokens).",
        )
    else:
        history = _set_assistant_status(history, "Summary gate not
needed. Proceeding with original script.")
    yield _yield_ui_state(history, payload, _status_html(routed_agent,
route_mode, stage="Script Ready"))

    history = _set_assistant_status(history, "Running Script Reviewer
model inference...")
    yield _yield_ui_state(history, payload, _status_html(routed_agent,
route_mode, stage="Script Reviewer Running"))

    answer_body, _, _ = evaluate_story(fit_result.get("story",
merged_submission), effective_mission_text, None)

    summary_meta = (
        f"**Summary Gate:** `applied` |
model=`{fit_result.get('summary_model') or 'n/a'}` | "
        f"prompt_tokens≈`{fit_result.get('prompt_tokens')}`"
        if fit_result.get("summary_applied")
        else f"**Summary Gate:** `not applied` |
prompt_tokens≈`{fit_result.get('prompt_tokens')}`"
    )

    if fit_result.get("error"):
        summary_meta += f"\n\n**Summary Note:**
{fit_result.get('error')}"

    final_answer = (
        f"**Routed Agent:** `{routed_agent}`\n\n"
        f"**Route Mode:** `{route_mode}`\n\n"
        f"{summary_meta}\n\n"
        f"{answer_body}"
    )

    history = _set_assistant_status(history, final_answer)
    yield _yield_ui_state(history, {}, _status_html(routed_agent,
route_mode, stage="Completed"))
```

```
def clear_chat_state():
    _persist_chat_history([])
    return [], "", [], {}, "", "", "No file attached.",
_status_html(stage="Idle")


CUSTOM_CSS = """
@import url('https://fonts.googleapis.com/css2?
family=Manrope:wght@400;500;600;700;800&family=IBM+Plex+Mono:wght@500&
display=swap');

:root {
    --ink: #0f2440;
    --muted: #5b6f88;
    --surface: #f8fbff;
    --surface-2: #ecf4fb;
    --surface-3: #d8e7f6;
    --line: rgba(38, 67, 104, 0.16);
    --line-strong: rgba(26, 58, 98, 0.28);
    --navy-900: #0b1d36;
    --navy-800: #122c4d;
    --blue-600: #0f71d7;
    --blue-500: #208bf0;
    --teal-400: #31b9c2;
    --warning: #f09e32;
    --ok: #1f8a4d;
}

.gradio-container {
    background:
        radial-gradient(70rem 40rem at -12% -20%, rgba(39, 143, 238,
0.24) 0%, rgba(39, 143, 238, 0) 58%),
        radial-gradient(65rem 36rem at 116% 4%, rgba(29, 182, 171,
0.22) 0%, rgba(29, 182, 171, 0) 56%),
        linear-gradient(165deg, #edf5fc 0%, #ddeaf7 46%, #d6e6f5 100%)
!important;
    font-family: 'Manrope', 'Avenir Next', 'Segoe UI', sans-serif !
important;
    color: var(--ink) !important;
}

#main-layout {
    max-width: 1320px;
    margin: 0 auto;
    min-height: 100vh;
    padding: 20px;
    gap: 16px;
}

.app-sidebar {
```

```
    background: linear-gradient(180deg, var(--navy-800) 0%, var(--
navy-900) 100%) !important;
    border: 1px solid rgba(180, 208, 236, 0.16) !important;
    border-radius: 20px !important;
    padding: 16px !important;
    box-shadow: 0 14px 40px rgba(8, 27, 50, 0.38);
    color: #eff7ff !important;
}

.brand-mark {
    font-size: 28px;
    line-height: 1.1;
    font-weight: 800;
    letter-spacing: 0.2px;
    color: #f7fbff;
}

.brand-sub {
    margin-top: 6px;
    margin-bottom: 16px;
    color: rgba(220, 236, 255, 0.82);
    font-size: 13px;
    letter-spacing: 0.3px;
    text-transform: uppercase;
}

.sidebar-btn button {
    width: 100%;
    height: 42px;
    border-radius: 12px !important;
    border: 1px solid rgba(220, 236, 255, 0.16) !important;
    background: rgba(224, 241, 255, 0.08) !important;
    color: #eef7ff !important;
    font-weight: 600;
    margin-bottom: 8px;
    transition: background 0.2s ease, transform 0.2s ease;
}

.sidebar-btn button:hover {
    background: rgba(224, 241, 255, 0.2) !important;
    transform: translateY(-1px);
}

.mission-wrap {
    margin-top: 8px;
    border-radius: 14px;
    background: rgba(228, 242, 255, 0.08);
    border: 1px solid rgba(227, 240, 255, 0.18);
    padding: 10px;
}
```

```
.mission-wrap label {
    color: #cbe2ff !important;
    font-weight: 700 !important;
}

.mission-wrap textarea {
    border-radius: 10px !important;
    border: 1px solid rgba(157, 191, 224, 0.32) !important;
    background: rgba(14, 34, 62, 0.5) !important;
    color: #edf6ff !important;
}

.mission-wrap textarea::placeholder {
    color: rgba(204, 224, 245, 0.7) !important;
}

.main-shell {
    background: linear-gradient(180deg, rgba(248, 252, 255, 0.95) 0%,
rgba(236, 245, 252, 0.95) 100%) !important;
    border: 1px solid var(--line) !important;
    border-radius: 24px !important;
    overflow: hidden;
    padding: 0 !important;
    box-shadow: 0 18px 44px rgba(19, 42, 73, 0.2);
}

.main-header {
    padding: 22px 24px 14px 24px;
    border-bottom: 1px solid var(--line);
    background: linear-gradient(145deg, rgba(255, 255, 255, 0.92) 0%,
rgba(239, 248, 255, 0.9) 100%);
}

.main-header .eyebrow {
    margin: 0;
    font-size: 12px;
    letter-spacing: 0.48px;
    text-transform: uppercase;
    color: #3a628d;
    font-weight: 700;
}

.main-header h1 {
    margin: 4px 0 0 0;
    font-size: 38px;
    line-height: 1.1;
    color: #103761;
    letter-spacing: -0.2px;
}
```

```
.main-header p {
    margin: 8px 0 0 0;
    color: var(--muted);
    font-size: 15px;
}

.status-panel {
    padding: 12px 18px 2px 18px;
}

.status-strip {
    display: grid;
    grid-template-columns: repeat(3, minmax(0, 1fr));
    gap: 10px;
}

.status-pill {
    display: flex;
    justify-content: space-between;
    align-items: center;
    border-radius: 12px;
    border: 1px solid var(--line);
    background: linear-gradient(160deg, #ffffff 0%, var(--surface-2)
100%);
    padding: 10px 12px;
    min-height: 42px;
}

.status-pill .k {
    font-size: 11px;
    text-transform: uppercase;
    letter-spacing: 0.42px;
    color: #557394;
    font-weight: 700;
}

.status-pill .v {
    font-family: 'IBM Plex Mono', ui-monospace, SFMono-Regular, Menlo,
monospace;
    font-size: 12px;
    color: #13385f;
    font-weight: 600;
    margin-left: 8px;
    text-align: right;
    word-break: break-word;
}

.chat-surface {
    padding: 14px 16px;
```

```
}

.chat-surface > .column {
    min-height: 54vh;
    max-height: 62vh;
    overflow: auto !important;
    padding: 14px !important;
    border-radius: 16px !important;
    border: 1px solid rgba(170, 198, 227, 0.25) !important;
    background:
        radial-gradient(45rem 22rem at 88% -8%, rgba(56, 169, 226,
0.16) 0%, rgba(56, 169, 226, 0) 56%),
        linear-gradient(180deg, #0f2745 0%, #112d4f 100%) !important;
}

.chat-stream {
    background: transparent !important;
}

.chat-stream .message {
    border-radius: 14px !important;
    border: 1px solid rgba(182, 210, 236, 0.2) !important;
    padding: 12px 14px !important;
    line-height: 1.55 !important;
}

.chat-stream .message.user {
    background: linear-gradient(145deg, rgba(62, 88, 124, 0.9) 0%,
rgba(76, 102, 137, 0.86) 100%) !important;
    color: #eff6ff !important;
}

.chat-stream .message.bot,
.chat-stream .message.assistant {
    background: linear-gradient(145deg, rgba(17, 49, 84, 0.94) 0%,
rgba(24, 60, 98, 0.9) 100%) !important;
    color: #eaf4ff !important;
}

.chat-stream .message p,
.chat-stream .message li,
.chat-stream .message span,
.chat-stream .message div {
    color: inherit !important;
}

.composer-wrap {
    border-top: 1px solid var(--line);
    background: linear-gradient(180deg, #f6fbff 0%, #eff6fd 100%);
    padding: 14px 16px;
```

```
}

.composer-row {
    align-items: center !important;
    gap: 8px !important;
    border: 1px solid var(--line-strong);
    border-radius: 16px;
    padding: 8px;
    background: #ffffff;
    box-shadow: inset 0 1px 0 rgba(255, 255, 255, 0.8);
}

.attach-btn button,
.send-btn button {
    height: 42px;
    border-radius: 11px !important;
    font-weight: 700;
}

.attach-btn button {
    min-width: 112px;
    border: 1px solid rgba(57, 101, 146, 0.22) !important;
    background: linear-gradient(145deg, #f6fbff 0%, #e9f4ff 100%) !
important;
    color: #204972 !important;
}

.attach-btn button:hover {
    filter: brightness(0.98);
}

.send-btn button {
    min-width: 112px;
    border: none !important;
    background: linear-gradient(135deg, var(--blue-600) 0%, var(--
blue-500) 58%, var(--teal-400) 100%) !important;
    color: #f6fbff !important;
    box-shadow: 0 8px 20px rgba(18, 114, 210, 0.28);
}

.send-btn button:hover {
    filter: brightness(1.03);
    transform: translateY(-1px);
}

.composer-input,
.composer-input .wrap,
.composer-input .block,
.composer-input .container {
    background: transparent !important;
```

```
}

.composer-input,
.composer-input * {
    color: var(--ink) !important;
}

.composer-input textarea {
    border: none !important;
    box-shadow: none !important;
    background: transparent !important;
    color: var(--ink) !important;
    caret-color: #154777 !important;
    font-size: 16px !important;
    font-weight: 500 !important;
}

.composer-input textarea::placeholder {
    color: #6a7f98 !important;
}

.file-row {
    margin-top: 8px;
    align-items: center !important;
    gap: 10px !important;
}

.file-status,
.file-status p,
.file-status span {
    margin: 0 !important;
    color: #204a74 !important;
    font-size: 14px !important;
    font-weight: 700 !important;
    opacity: 1 !important;
    word-break: break-word;
}

.clear-file-btn button {
    height: 36px;
    border-radius: 10px !important;
    border: 1px solid rgba(63, 102, 145, 0.24) !important;
    background: #ffffff !important;
    color: #2e557e !important;
    font-weight: 600;
}

.clear-file-btn button:hover {
    background: #f2f8ff !important;
}
```

```
[data-testid="status-bar"],
.progress-text,
.meta-text,
.eta-bar,
.pending {
    display: none !important;
}

@keyframes uiFade {
    from { opacity: 0; transform: translateY(8px); }
    to { opacity: 1; transform: translateY(0); }
}

.main-shell,
.app-sidebar {
    animation: uiFade 320ms ease-out both;
}

@media (max-width: 1080px) {
    #main-layout {
        padding: 12px;
        gap: 12px;
    }

    .main-header h1 {
        font-size: 32px;
    }

    .status-strip {
        grid-template-columns: 1fr;
    }

    .chat-surface > .column {
        min-height: 52vh;
        max-height: 58vh;
    }
}
"""


_gradio_version = getattr(gr, "__version__", "0")
try:
    _gradio_major = int(str(_gradio_version).split(".")[0])
except Exception:
    _gradio_major = 0

if _gradio_major >= 6:
    _blocks_kwargs = {"fill_height": True}
    UI_LAUNCH_STYLE_KW = {"css": CUSTOM_CSS, "theme":
```

```
gr.themes.Base()}
else:
    _blocks_kwargs = {"css": CUSTOM_CSS, "theme": gr.themes.Base(),
"fill_height": True}
    UI_LAUNCH_STYLE_KW = {}

_initial_chat_history = _load_persisted_chat_history()

with gr.Blocks(**_blocks_kwargs) as demo:
    uploaded_text_state = gr.State("")
    uploaded_path_state = gr.State("")
    chat_state = gr.State(list(_initial_chat_history))
    pending_payload_state = gr.State({})

    with gr.Row(elem_id="main-layout"):
        with gr.Column(scale=0, min_width=270, elem_classes="app-
sidebar"):
            gr.HTML(
                """
                <div class="brand-mark">Impact Studios</div>
                <div class="brand-sub">Creative Impact Workspace</div>
                """
            )

            new_chat_btn = gr.Button("New chat",
elem_classes="sidebar-btn")
            search_btn = gr.Button("Search chat",
elem_classes="sidebar-btn")
            add_agent_btn = gr.Button("Add agent",
elem_classes="sidebar-btn")

            with gr.Column(elem_classes="mission-wrap"):
                mission_box = gr.Textbox(
                    label="Studio Mission (optional)",
                    placeholder="Add mission context for this run...",
                    lines=6,
                )

        with gr.Column(scale=1, elem_classes="main-shell"):
            gr.HTML(
                """
                <div class="main-header">
                    <p class="eyebrow">Orchestrated Creative
Intelligence</p>
                    <h1>Impact Studios Console</h1>
                    <p>Unified workspace for routing, script review,
and impact analysis.</p>
                </div>
                """
            )
```

```
            status_panel = gr.HTML(_status_html(stage="Idle"),
elem_classes="status-panel")

            with gr.Column(elem_classes="chat-surface"):
                _chatbot_sig =
set(inspect.signature(gr.Chatbot.__init__).parameters.keys())
                _chatbot_kwargs = {
                    "value": list(_initial_chat_history),
                    "show_label": False,
                    "container": False,
                    "bubble_full_width": False,
                    "elem_classes": "chat-stream",
                }
                if "type" in _chatbot_sig:
                    _chatbot_kwargs["type"] = "messages"
                elif "message_type" in _chatbot_sig:
                    _chatbot_kwargs["message_type"] = "messages"

                chatbot = gr.Chatbot(**{k: v for k, v in
_chatbot_kwargs.items() if k in _chatbot_sig})

            with gr.Column(elem_classes="composer-wrap"):
                with gr.Row(elem_classes="composer-row",
equal_height=True):
                    upload_btn = gr.UploadButton(
                        "Attach",
                        file_types=[".txt", ".pdf", ".docx", ".doc",
".md"],
                        file_count="single",
                        elem_classes="attach-btn",
                        scale=0,
                    )

                    story = gr.Textbox(
                        label="",
                        placeholder="Ask anything",
                        lines=1,
                        max_lines=5,
                        show_label=False,
                        container=False,
                        elem_classes="composer-input",
                        scale=1,
                    )

                    submit_btn = gr.Button("Send", elem_classes="send-
btn", scale=0)

                with gr.Row(elem_classes="file-row"):
                    with gr.Column(scale=1, min_width=220):
```

```
                        file_status = gr.Markdown("No file attached.",
elem_classes="file-status")
                    with gr.Column(scale=0, min_width=140):
                        clear_file_btn = gr.Button("Clear file",
elem_classes="clear-file-btn")

    upload_btn.upload(
        fn=on_upload,
        inputs=upload_btn,
        outputs=[uploaded_text_state, file_status,
uploaded_path_state],
        show_progress="hidden",
    )

    clear_file_btn.click(
        fn=clear_uploaded_state,
        inputs=None,
        outputs=[uploaded_text_state, file_status,
uploaded_path_state],
        show_progress="hidden",
    )

    submit_event = submit_btn.click(
        fn=stage_user_message,
        inputs=[story, mission_box, uploaded_text_state,
uploaded_path_state, chat_state],
        outputs=[chatbot, story, chat_state, pending_payload_state,
status_panel],
        show_progress="hidden",
    )

    submit_event.then(
        fn=resolve_assistant_message,
        inputs=[chat_state, pending_payload_state],
        outputs=[chatbot, chat_state, pending_payload_state,
status_panel],
        show_progress="hidden",
    )

    enter_event = story.submit(
        fn=stage_user_message,
        inputs=[story, mission_box, uploaded_text_state,
uploaded_path_state, chat_state],
        outputs=[chatbot, story, chat_state, pending_payload_state,
status_panel],
        show_progress="hidden",
    )

    enter_event.then(
        fn=resolve_assistant_message,
```

```
        inputs=[chat_state, pending_payload_state],
        outputs=[chatbot, chat_state, pending_payload_state,
status_panel],
        show_progress="hidden",
    )

    new_chat_btn.click(
        fn=clear_chat_state,
        inputs=None,
        outputs=[
            chatbot,
            story,
            chat_state,
            pending_payload_state,
            uploaded_text_state,
            uploaded_path_state,
            file_status,
            status_panel,
        ],
        show_progress="hidden",
    )

print("[STEP 14] Unified UI object is ready as variable: demo")
```

```
[STEP 14] Unified UI object is ready as variable: demo
```

NEXT MANUAL: Launch UI

After this section, run the next cell to launch Gradio and open the printed link.

How To Test The System

1. Click **Attach** and upload a local script file (`.pdf` recommended).
2. In the input box, type a prompt. Your prompt is used by the orchestrator to route the task.
3. Press **Enter** or click **Send** to start analysis.
4. For a different script file, click **Clear attached file**, then upload a new one.

Example Prompt (Impact Agent, clearer route)

```
Please run an impact analysis: identify community impact, social
outcomes, ethical risks, policy implications, and harm-reduction
outreach actions for affected communities.
```

Example Prompt (Script Reviewer, clearer route)

```
Please review this script and tell me how I can improve it
```

Interaction Notes

- Multi-turn chat messages are treated as independent requests for analysis.

- File upload is separate from text prompt; routing uses your current prompt content.

```
# STEP 14.1: Launch Gradio UI
if "demo" not in globals():
    raise RuntimeError("demo is not defined. Please run STEP 14
first.")

launch_kwargs = {"share": True, "debug": True}
if isinstance(globals().get("UI_LAUNCH_STYLE_KW"), dict):
    launch_kwargs.update(UI_LAUNCH_STYLE_KW)

print("[STEP 14.1] Launching Gradio with kwargs:", launch_kwargs)
demo.launch(**launch_kwargs)
```

```
[STEP 14.1] Launching Gradio with kwargs: {'share': True, 'debug':
True, 'css': '\n@import url(\'https://fonts.googleapis.com/css2?
family=Manrope:wght@400;500;600;700;800&family=IBM+Plex+Mono:wght@500&
display=swap\');\n\n:root {\n    --ink: #0f2440;\n    --muted:
#5b6f88;\n    --surface: #f8fbff;\n    --surface-2: #ecf4fb;\n    --
surface-3: #d8e7f6;\n    --line: rgba(38, 67, 104, 0.16);\n    --line-
strong: rgba(26, 58, 98, 0.28);\n    --navy-900: #0b1d36;\n    --navy-
800: #122c4d;\n    --blue-600: #0f71d7;\n    --blue-500: #208bf0;\n
--teal-400: #31b9c2;\n    --warning: #f09e32;\n    --ok: #1f8a4d;\n}\
n\n.gradio-container {\n    background:\n        radial-gradient(70rem
40rem at -12% -20%, rgba(39, 143, 238, 0.24) 0%, rgba(39, 143, 238, 0)
58%),\n        radial-gradient(65rem 36rem at 116% 4%, rgba(29, 182,
171, 0.22) 0%, rgba(29, 182, 171, 0) 56%),\n        linear-
gradient(165deg, #edf5fc 0%, #ddeaf7 46%, #d6e6f5 100%) !important;\n
font-family: \'Manrope\', \'Avenir Next\', \'Segoe UI\', sans-serif !
important;\n    color: var(--ink) !important;\n}\n\n#main-layout {\n
max-width: 1320px;\n    margin: 0 auto;\n    min-height: 100vh;\n
padding: 20px;\n    gap: 16px;\n}\n\n.app-sidebar {\n    background:
linear-gradient(180deg, var(--navy-800) 0%, var(--navy-900) 100%) !
important;\n    border: 1px solid rgba(180, 208, 236, 0.16) !
important;\n    border-radius: 20px !important;\n    padding: 16px !
important;\n    box-shadow: 0 14px 40px rgba(8, 27, 50, 0.38);\n
color: #eff7ff !important;\n}\n\n.brand-mark {\n    font-size: 28px;\n
line-height: 1.1;\n    font-weight: 800;\n    letter-spacing: 0.2px;\n
color: #f7fbff;\n}\n\n.brand-sub {\n    margin-top: 6px;\n    margin-
bottom: 16px;\n    color: rgba(220, 236, 255, 0.82);\n    font-size:
13px;\n    letter-spacing: 0.3px;\n    text-transform: uppercase;\n}\
n\n.sidebar-btn button {\n    width: 100%;\n    height: 42px;\n
border-radius: 12px !important;\n    border: 1px solid rgba(220, 236,
255, 0.16) !important;\n    background: rgba(224, 241, 255, 0.08) !
important;\n    color: #eef7ff !important;\n    font-weight: 600;\n
margin-bottom: 8px;\n    transition: background 0.2s ease, transform
0.2s ease;\n}\n\n.sidebar-btn button:hover {\n    background:
rgba(224, 241, 255, 0.2) !important;\n    transform: translateY(-
1px);\n}\n\n.mission-wrap {\n    margin-top: 8px;\n    border-radius:
14px;\n    background: rgba(228, 242, 255, 0.08);\n    border: 1px
```

solid rgba(227, 240, 255, 0.18);\n   padding: 10px;\n}\n\n.mission-wrap label {\n   color: #cbe2ff !important;\n   font-weight: 700 !important;\n}\n\n.mission-wrap textarea {\n   border-radius: 10px !important;\n   border: 1px solid rgba(157, 191, 224, 0.32) !important;\n   background: rgba(14, 34, 62, 0.5) !important;\n color: #edf6ff !important;\n}\n\n.mission-wrap textarea::placeholder {\n   color: rgba(204, 224, 245, 0.7) !important;\n}\n\n.main-shell {\n   background: linear-gradient(180deg, rgba(248, 252, 255, 0.95) 0%, rgba(236, 245, 252, 0.95) 100%) !important;\n   border: 1px solid var(--line) !important;\n   border-radius: 24px !important;\n overflow: hidden;\n   padding: 0 !important;\n   box-shadow: 0 18px 44px rgba(19, 42, 73, 0.2);\n}\n\n.main-header {\n   padding: 22px 24px 14px 24px;\n   border-bottom: 1px solid var(--line);\n background: linear-gradient(145deg, rgba(255, 255, 255, 0.92) 0%, rgba(239, 248, 255, 0.9) 100%);\n}\n\n.main-header .eyebrow {\n margin: 0;\n   font-size: 12px;\n   letter-spacing: 0.48px;\n text-transform: uppercase;\n   color: #3a628d;\n   font-weight: 700;\n}\n\n.main-header h1 {\n   margin: 4px 0 0 0;\n   font-size: 38px;\n   line-height: 1.1;\n   color: #103761;\n   letter-spacing: -0.2px;\n}\n\n.main-header p {\n   margin: 8px 0 0 0;\n   color: var(--muted);\n   font-size: 15px;\n}\n\n.status-panel {\n padding: 12px 18px 2px 18px;\n}\n\n.status-strip {\n   display: grid;\n   grid-template-columns: repeat(3, minmax(0, 1fr));\n   gap: 10px;\n}\n\n.status-pill {\n   display: flex;\n   justify-content: space-between;\n   align-items: center;\n   border-radius: 12px;\n border: 1px solid var(--line);\n   background: linear-gradient(160deg, #ffffff 0%, var(--surface-2) 100%);\n   padding: 10px 12px;\n   min-height: 42px;\n}\n\n.status-pill .k {\n   font-size: 11px;\n   text-transform: uppercase;\n   letter-spacing: 0.42px;\n   color: #557394;\n   font-weight: 700;\n}\n\n.status-pill .v {\n   font-family: \'IBM Plex Mono\', ui-monospace, SFMono-Regular, Menlo, monospace;\n   font-size: 12px;\n   color: #13385f;\n   font-weight: 600;\n   margin-left: 8px;\n   text-align: right;\n   word-break: break-word;\n}\n\n.chat-surface {\n   padding: 14px 16px;\n}\n\n.chat-surface > .column {\n   min-height: 54vh;\n   max-height: 62vh;\n   overflow: auto !important;\n   padding: 14px !important;\n   border-radius: 16px !important;\n   border: 1px solid rgba(170, 198, 227, 0.25) !important;\n   background:\n radial-gradient(45rem 22rem at 88% -8%, rgba(56, 169, 226, 0.16) 0%, rgba(56, 169, 226, 0) 56%),\n   linear-gradient(180deg, #0f2745 0%, #112d4f 100%) !important;\n}\n\n.chat-stream {\n   background: transparent !important;\n}\n\n.chat-stream .message {\n   border-radius: 14px !important;\n   border: 1px solid rgba(182, 210, 236, 0.2) !important;\n   padding: 12px 14px !important;\n   line-height: 1.55 !important;\n}\n\n.chat-stream .message.user {\n   background: linear-gradient(145deg, rgba(62, 88, 124, 0.9) 0%, rgba(76, 102, 137, 0.86) 100%) !important;\n   color: #eff6ff !important;\n}\n\n.chat-stream .message.bot,\n.chat-stream .message.assistant {\n background: linear-gradient(145deg, rgba(17, 49, 84, 0.94) 0%,

rgba(24, 60, 98, 0.9) 100%) !important;\n   color: #eaf4ff !important;\n}\n\n.chat-stream .message p,\n.chat-stream .message li,\n.chat-stream .message span,\n.chat-stream .message div {\n   color: inherit !important;\n}\n\n.composer-wrap {\n   border-top: 1px solid var(--line);\n   background: linear-gradient(180deg, #f6fbff 0%, #eff6fd 100%);\n   padding: 14px 16px;\n}\n\n.composer-row {\n   align-items: center !important;\n   gap: 8px !important;\n   border: 1px solid var(--line-strong);\n   border-radius: 16px;\n   padding: 8px;\n   background: #ffffff;\n   box-shadow: inset 0 1px 0 rgba(255, 255, 255, 0.8);\n}\n\n.attach-btn button,\n.send-btn button {\n   height: 42px;\n   border-radius: 11px !important;\n   font-weight: 700;\n}\n\n.attach-btn button {\n   min-width: 112px;\n   border: 1px solid rgba(57, 101, 146, 0.22) !important;\n   background: linear-gradient(145deg, #f6fbff 0%, #e9f4ff 100%) !important;\n   color: #204972 !important;\n}\n\n.attach-btn button:hover {\n   filter: brightness(0.98);\n}\n\n.send-btn button {\n   min-width: 112px;\n   border: none !important;\n   background: linear-gradient(135deg, var(--blue-600) 0%, var(--blue-500) 58%, var(--teal-400) 100%) !important;\n   color: #f6fbff !important;\n   box-shadow: 0 8px 20px rgba(18, 114, 210, 0.28);\n}\n\n.send-btn button:hover {\n   filter: brightness(1.03);\n   transform: translateY(-1px);\n}\n\n.composer-input,\n.composer-input .wrap,\n.composer-input .block,\n.composer-input .container {\n   background: transparent !important;\n}\n\n.composer-input,\n.composer-input * {\n   color: var(--ink) !important;\n}\n\n.composer-input textarea {\n   border: none !important;\n   box-shadow: none !important;\n   background: transparent !important;\n   color: var(--ink) !important;\n   caret-color: #154777 !important;\n   font-size: 16px !important;\n   font-weight: 500 !important;\n}\n\n.composer-input textarea::placeholder {\n   color: #6a7f98 !important;\n}\n\n.file-row {\n   margin-top: 8px;\n   align-items: center !important;\n   gap: 10px !important;\n}\n\n.file-status,\n.file-status p,\n.file-status span {\n   margin: 0 !important;\n   color: #204a74 !important;\n   font-size: 14px !important;\n   font-weight: 700 !important;\n   opacity: 1 !important;\n   word-break: break-word;\n}\n\n.clear-file-btn button {\n   height: 36px;\n   border-radius: 10px !important;\n   border: 1px solid rgba(63, 102, 145, 0.24) !important;\n   background: #ffffff !important;\n   color: #2e557e !important;\n   font-weight: 600;\n}\n\n.clear-file-btn button:hover {\n   background: #f2f8ff !important;\n}\n\n[data-testid="status-bar"],\n.progress-text,\n.meta-text,\n.eta-bar,\n.pending {\n   display: none !important;\n}\n\n@keyframes uiFade {\n   from { opacity: 0; transform: translateY(8px); }\n   to { opacity: 1; transform: translateY(0); }\n}\n\n.main-shell,\n.app-sidebar {\n   animation: uiFade 320ms ease-out both;\n}\n\n@media (max-width: 1080px) {\n   #main-layout {\n   padding: 12px;\n   gap: 12px;\n   }\n\n   .main-header h1 {\n   font-size: 32px;\n   }\n\n   .status-strip {\n   grid-template-columns: 1fr;\n   }\n\n   .chat-surface > .column {\n   min-height: 52vh;\n

```
max-height: 58vh;\n    }\n}\n', 'theme': <gradio.themes.base.Base
object at 0x798b1bf9e780>}
Colab notebook detected. This cell will run indefinitely so that you
can see errors and logs. To turn off, set debug=False in launch().
* Running on public URL: https://bc387783b193c72085.gradio.live

This share link expires in 1 week. For free permanent hosting and GPU
upgrades, run `gradio deploy` from the terminal in the working
directory to deploy to Hugging Face Spaces
(https://huggingface.co/spaces)

<IPython.core.display.HTML object>

WARNING:pypdf._reader:Ignoring wrong pointing object 5 0 (offset 0)


==========================================================================
==============================
[MODEL RAW OUTPUT] script-reviewer
==========================================================================
==============================
Rating Reasoning:
This uplifting story explores redemption and responsibility across
generations.

Verdict: Yes
Score: 5/5
Benefits:
- It deepens empathy and understanding between strangers.
- It models accountability and growth.
- It highlights the importance of human connection.
- It invites reflection on past choices and future possibilities.
- It offers hope for healing and reconciliation.
Risks:
- None significant.
Overall rationale: This narrative uplifts humanity by showing how
confronting the past can lead to forgiveness and new beginnings.
==========================================================================
==============================


==========================================================================
==============================
[MODEL RAW OUTPUT] script-reviewer
==========================================================================
==============================
...nings. It centers compassion and encourages readers to consider the
complexities of real-life relationships.
==========================================================================
==============================
```

```
Keyboard interruption in main thread... closing server.
Killing tunnel 127.0.0.1:7860 <>
https://bc387783b193c72085.gradio.live
```

Appendix Notes

- Existing code blocks are intentionally retained to preserve behavior parity with v1.
- Chat History is not added at this point since the model cannot run it through. Discuss during meeting time